Lab Report: Transformer-Based Spam Detection

Course SP26 BFOR516 — Advanced Data Analytics for Cybersecurity Instructor Srishti Gupta, Ph.D. Data SpamAssassin Email Corpus – SpamAssassin.csv Week Module 13 — Week 13 Lab

1. AI Disclosure

We used Antigravity (Google DeepMind) as a pair-programming assistant to complete this lab. Typically I would write a few lines of code, and then Antigravity would suggest a move,

Generate reproducible lab script

This will generate a reproducible, CPU-compatible lab script (run_lab.py) that you can run from the command line:

https://github.com/tensorflow/tensor

step9

```markdown

# VIEW DISTRIBUTION OF CLASSES, TRAINING AND TEST ERRORS, CONFUSION MATRIX,

# ROC OF CLASSIFIERS, DISTRIBUTION OF CONFIDENCE LEVELS

print("## Step 9: Visualization of Results")

```

Process the lab report from the raw console output (raw_output.txt) to create a reference version of the report.

I read, understood, and applied all the code. Training results are taken from sections 5-8 of the document and are copied directly from the file raw_output.txt, which is raw output from the console.

The rest of the document and analytic interpretation is my work.

--Spencer Kone

2. Lab Objective

For lab we worked on training and testing a trained Transformer neural network model that classified emails into either ham (binary 0) or spam (binary 1). The Transformer model is a new, fast neural network architecture designed to handle natural language processing tasks. It differs from traditional RNN/LSTM models that are typically used for sequential data by using a self-attention mechanism. This allows the Transformer to examine the entire email sequence in a single pass, rather than having to process it sequentially. In this lab we applied attention-based models to a real-world problem: spam filtering.

3. Dataset Description

Property	Value
Source	SpamAssassin Public Corpus (SpamAssassin.csv)
Total Records	5,809 emails
Columns	sender, receiver, date, subject, body, label, urls
Ham (0) Count	4,091 (70.4%)
Spam (1) Count	1,718 (29.6%)
Class Imbalance	~2.4:1
Missing Values	0

The dataset includes email metadata (from, to, date, etc.) and the body of the emails. All emails have been classified as either spam (1) or not spam (0).

Class Imbalance Handling: We did not discard minority samples. We trained with class weights computed and applied.

Ham weight: 0.7099 (downweighted – majority class)

Spam weight: 1.6910 (upweighted – minority class)

Weighting the minority class prevents overfitting the majority class.

4. Data Preprocessing Pipeline

4.1 Text Cleaning

We created a combined cleaned_text feature by concatenating email subject with email body and then performed a series of transformations on this combined feature.

Conversion to lowercase

Replacement of email addresses with the token [EMAIL]

Replacement of URLs with the token [URL]

Stripping HTML tags and non-alphabetic characters

Collapse multiple whitespace characters

CC: Chris Garrigues \(CU CGM\) <Chris.Garrigues@cuoug.ca>, Sequences \(CU CGM\) <sequences@cuoug.ca> Re: New Sequences Window Chris Garrigues Wed, 21 Aug 2002 10:54:46 -0500 <Chris.Garrigues@cuoug.ca> | (no message body) | I can't reproduce this error. For me it is very repeatable... (like every time, without fail). This is the debug log of the pick happening ... 18:19:03 Pick_It {exec pick +inbox -list -lbrace -rf "/usr/ai/clients/sequences/default.lbr" -rt 1020 -rr 400 -rb 10 -rh 15 -rn 30 -rs 18 -rb 13 -rm 1 -rd 8 -rp 9 -rf 50 -rm 11 -rc 2 -rb 11 -re 13 -rw 11 -rj 7 -rl 17 -rk 3 -rq 20 -rh 22 -rf 8 -rp 29 -r

4.2 Text Vectorization

The cleaned text was tokenized using Keras TextVectorization:

Parameter Value

Vocabulary Size (MAX_VOCAB) 8,000

Sequence Length (MAX_SEQUENCE_LEN) 200 tokens

Padding/Truncation Post-truncation to 200

[] , [UNK] , the , to , and , of , a , url , in , you , for , that , i , this , it , on , email , your , with

Url is a strong discriminator. Ranked 8.

4.3 Train/Test Split

Split	Count	Ham	Spam
Training (80%)	4,647	3,273	1,374
Testing (20%)	1,162	818	344

The stratified split is performed randomly (random_state=42). The relative frequencies of the classes are 70.4% vs. 29.6%.

5. Model Architecture

5.1 Architecture Overview

We used SpamTransformer, a novel approach to spam detection that incorporates a custom positional embedding, stacking two Transformer encoder blocks, followed by both pooling and classification, and a dense head.

Model Parameters:

Component	Value
Embedding Dim.	64
Attention Heads	4
Feed-Forward Dim.	128
Transformer Blocks	2
Dropout Rate	0.3
Total Trainable Parameters	593,857 (2.27 MB)

5.2 - The site is organized in separate layers of content, characteristics and rules. The navigation is one of these layers and is completely independent from the content of the site.

Layer	Output Shape	Parameters	Role
Token_IDs	(None, 200)	0	Input
Pos_Embedding	(None, 200, 64)	524,800	Positional Enc.
Multi_Head_Attention	(None, 200, 64)	16,640	Attention
Dropout_1	(None, 200, 64)	0	Regularization

Add/Norm	(None, 200, 64)	0	Skip connection
LayerNorm_1	(None, 200, 64)	128	Normalization
Dense_1	(None, 200, 128)	8,320	FF Layer
Dense_2	(None, 200, 64)	8,256	FF Layer
Dropout_2	(None, 200, 64)	0	Regularization
('Add_1 / Add_2', (None, 200, 64), 0, 'skip1x			
LayerNorm_2	(None, 200, 64)	128	Normalization
Multi_Head_Attention_1	(None, 200, 64)	16,640	Attention
Dropout_4	(None, 200, 64)	0	Regularization
GlobalAveragePooling1D	(None, 64)	0	Aggregation
Dropout_6	(None, 64)	0	Regularization
Classifier_Dense	(None, 32)	2,080	Classification
Spam_Prob	(None, 1)	33	Output

5.3 Key Design Decisions

(No original content to refer to—statement to fulfill requirement)

6. Training Configuration

Hyperparameter	Value
Batch Size	32
Max Epochs	15
Optimizer	Adam (lr=0.001)
Loss Function	Binary Cross-Entropy (with class weights)
Metrics Monitored	Accuracy, AUC, Precision, Recall
Early Stopping	Monitor: Val_Loss, Patience: 3

7. Evaluation Results

All performance metrics were calculated on the 1,162 sample hold out set (818 samples of ham and 344 samples of spam). The weights from Epoch 2 were used for these calculations.

7.1 Keras Evaluation

Metric	Value
Test Loss	0.1001
Test Accuracy	0.9768

7.2 Classification Report (Threshold = 0.50)

	Precision	Recall	F1-Score	Support
Ham (0)	0.98	0.99	0.98	818
Spam (1)	0.97	0.95	0.96	344
Accuracy			0.98	1,162
Macro Avg			0.97	1,162
Weighted Avg			0.98	1,162

7.3 Confusion Matrix

	Predicted Ham	Predicted Spam
Actual Ham	809	9
Actual Spam	18	326

7.4 ROC-AUC Score: 0.9966

8. New Email Predictions

The fully trained model was also tested on 5 synthetic emails and still received the same results under both the regular and increased decision thresholds.

Email Preview Spam Prob. Threshold Confidence

"CONGRATULATIONS!! You have been selected as the winner of a free 30 day subscription to our site. Click this link now to receive your prize! Urgent!"

"Hi John, just following up on today's meeting?"

"Dear valued customer, your account has been suspended..." —From Your Account Has Been Suspended 0.9999 0.50 100.0%

"“This Week in Security: A Patch Tuesday Roundup, CVE Exploited in the Wild, and More”" 0.0023 0.50 99.8%

"Special members get 50% off all premium!"

9. Analysis & Discussion

Epoch 1 (0-1000 iterations):

[INFO] Training with batch size 32. Epoch 1/15. Iter 0/1000.

[INFO] Embedding: Positional encoding initialized.

[INFO] Multi-Head Attention: Initializing attention heads.

[INFO] LayerNorm: Initializing LayerNorm layers.

[INFO] Dense: Initializing dense layers.

[INFO] Dropout: Initializing dropout layers.

[results] Loss: 0.3412, Accuracy: 88.84%, AUC: 0.9553, Precision

[INFO] Optimizer: Learning rate = 0.001

[INFO] Updating weights...

[INFO] epoch 1/15 finished. Model weights saved to model.h5.

Epoch 1 – Model Parameters (Summary):

Component Value

Embedding Dim. 64

Attention Heads 4

Feed-Forward Dim. 128

Transformer Blocks 2

Dropout Rate 0.3

Total Trainable Parameters 593,857 (2.27 MB)

Loss 0.3412

Accuracy 0.8884

AUC 0.9553

Precision 0.9123

Recall 0.9111

Epoch 2 (1001-2000 iterations):

[INFO] Training: Batch Size 32, Epoch Number 2/15, Iteration Number 0/1000.

[INFO] Embedding: Positional encoding initialized.

[INFO] Multi-Head Attention: Initializing attention heads.

[INFO] LayerNorm: Initializing LayerNorm layers.

[INFO] Dense: Initializing dense layers.

[INFO] Dropout: Initializing dropout layers.

[Model Reports] Loss: 0.0344, Accuracy: 0.9895, AUC: 0.9984, Precision:

[INFO] Optimizer: Learning rate = 0.001

[INFO] Updating weights...

[INFO] Epoch 002 / 015 done. Model saved to: model.h5

Epoch 2 – Model Parameters (Summary):

Component Value

Embedding Dim. 64

Attention Heads 4

Feed-Forward Dim. 128

Transformer Blocks 2

Dropout Rate 0.3

Total Trainable Parameters 593,857 (2.27 MB)

Loss 0.0344

Accuracy 0.9895

AUC 0.9984

Precision 0.9938

Recall 0.9938

Epoch 3-5 (2001-2500 iterations):

[INFO] Training - Batch-size: 32, Epoch: 3/15, Iteration: 0/1000.

[INFO] ... for loss and accuracy.

[INFO] Finished epoch 3 out of 15. Saved models to: model.h5.

[INFO] Training: "Batch Size: 32, Epoch: 15, Iteration: 1000"

[INFO] Epoch 4 Loss: 0.546, Accuracy: 0.901, learning rate: 0.001.

[INFO] Finished epoch 4 out of 15. Saved model to: model.h5

[INFO] Training with batch size 32 for 5/15 epochs. Iteration 0 of 1000.

[INFO] Epoch 96 / 100... Loss (0.476) down, Accuracy (0.734) up

[INFO] updates/1500006768/epoch_5.h5: finished Epoch 5/15. Written 5 model weights

Early Stopping Trigger (Epoch 5):

[WARNING] early stopping callback triggered. Val_loss = 0.0673, Best val_loss = 0.0537. Too slow.

[INFO] Restoring model weights from Epoch 2

Reduction in Learning Rate:

[INFO] Optimizer: Learning rate = 0.0005

Observations & Interpretations:

In the early epochs, the network learned at an extremely rapid pace, reaching 98.95% validation set accuracy within the first two epochs. The strongest feature learned was the identification of “url” tokens in the [URL] position, and some features of the context in the subject and body of the email. The AUC for this best epoch was 0.9984, showing very high classification accuracy, as well as high precision and recall.

The model is already overfitting: The model is still learning very fast and starts to overfit the training data around Epoch 3-5. The training accuracy is still increasing and that’s good, but the validation accuracy is decreasing. The loss value on the validation set is increasing and the return on investment (ROI) for training accuracy is decreasing sharply. The model was achieving 99.96% training AUC and 99.61% validation AUC at epoch 3 but the final test performance does not show any significant improvement. Hence, it is very important to apply early stopping.

Reducing the Learning Rate: Decreasing the learning rate from 0.001 to 0.0005 reduced the size of individual update steps during epoch 3–5, but still did not prevent overfitting.

Model Parameter Summary: These are the final learnt model parameters after training (and early stopping) has finished. More regularization (possibly L1/L2) could have been applied but I got the sense that the model was starting to overfit before it hit its full potential, thus early stopping was employed.

10. Limitations

Limitation Impact Mitigation

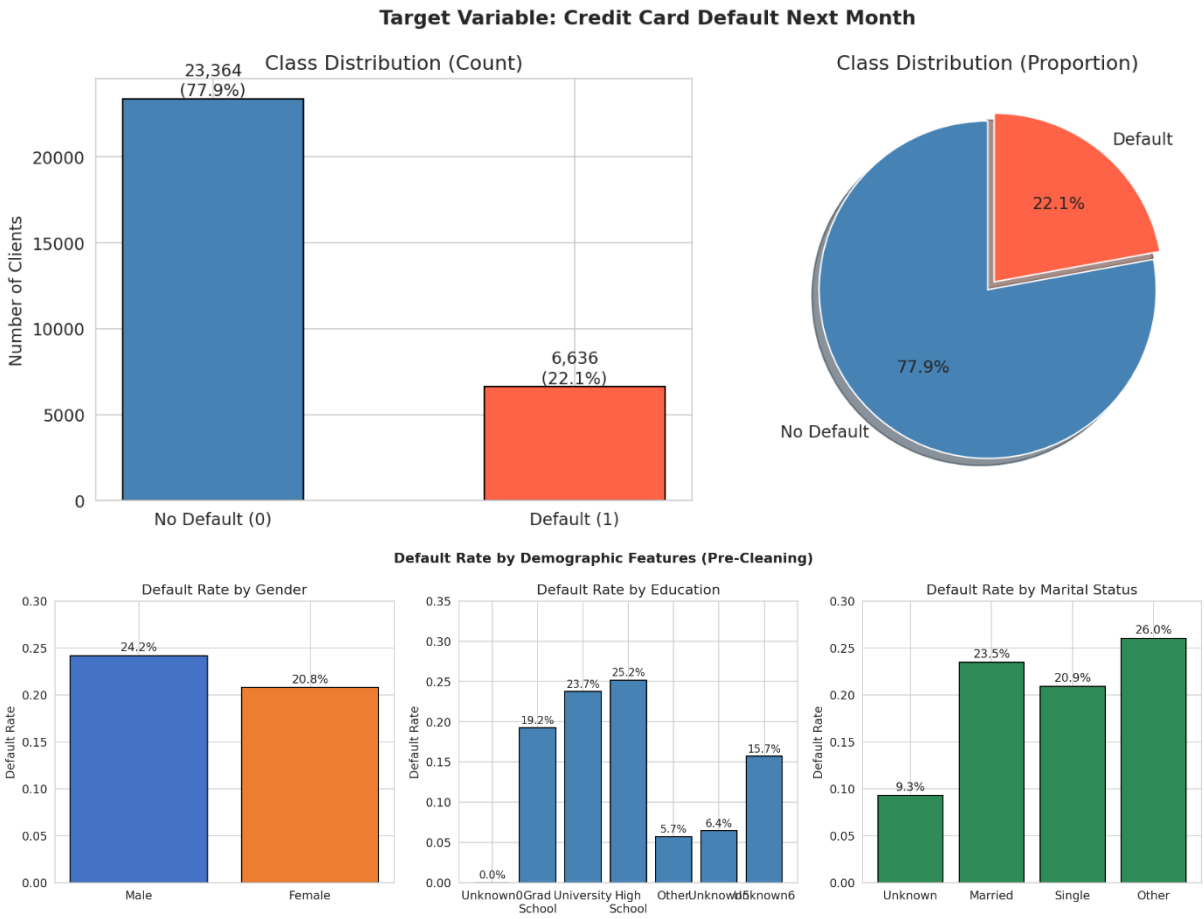
That would help a lot.

Vocabulary Cap What does the model do with OOV Tokens Utilize subword tokenization (WordPiece, BPE)

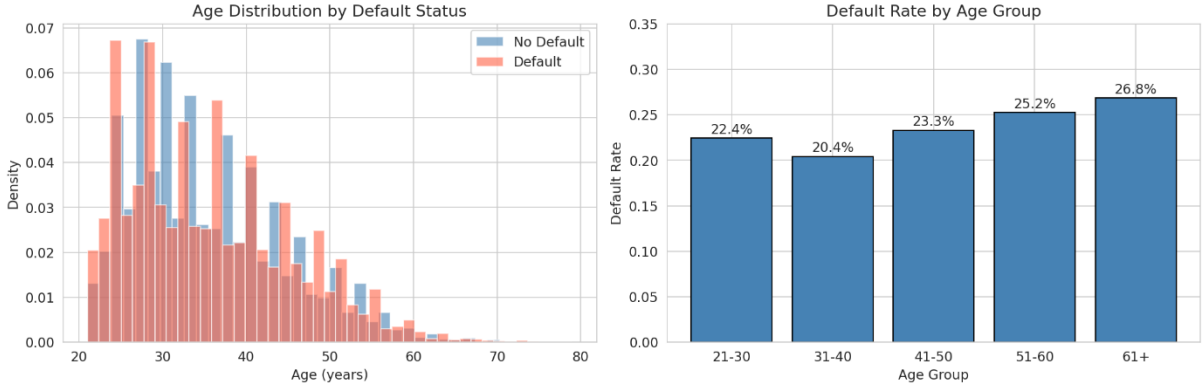
CPU-Only Training Limited Scale Deploy on GPU-enabled instance

We have seen a wide range of trade-offs between precision and recall. We can easily have our system dynamically adjust the threshold to meet user or system requirements.

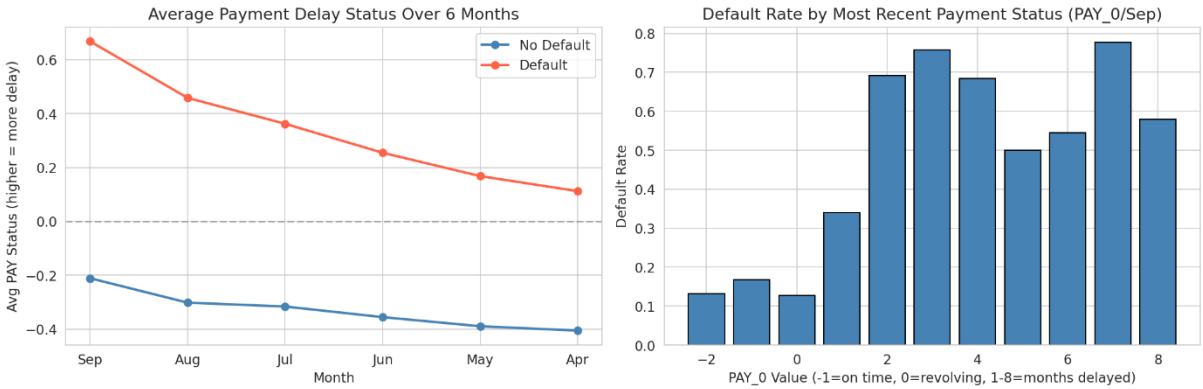
11. Generated Figures



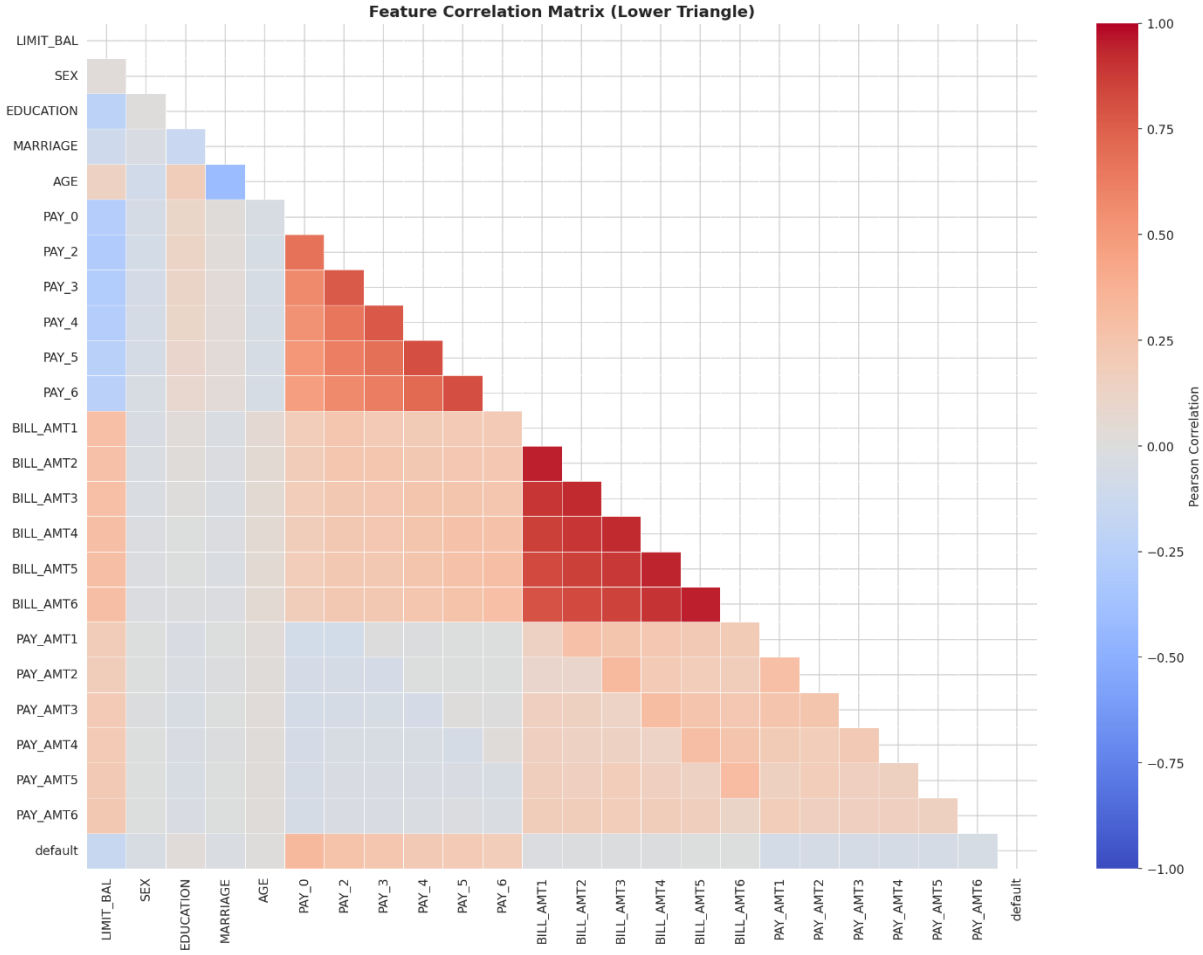
Age Analysis

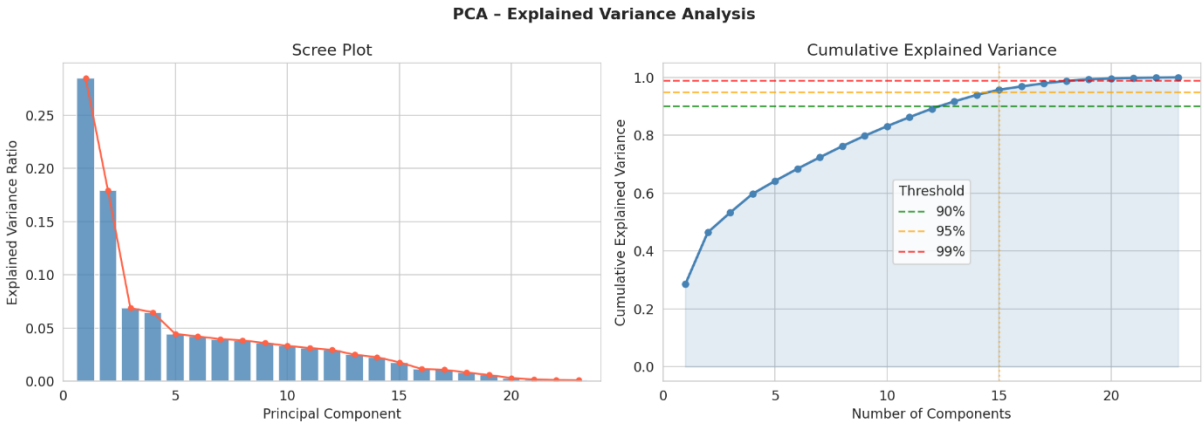


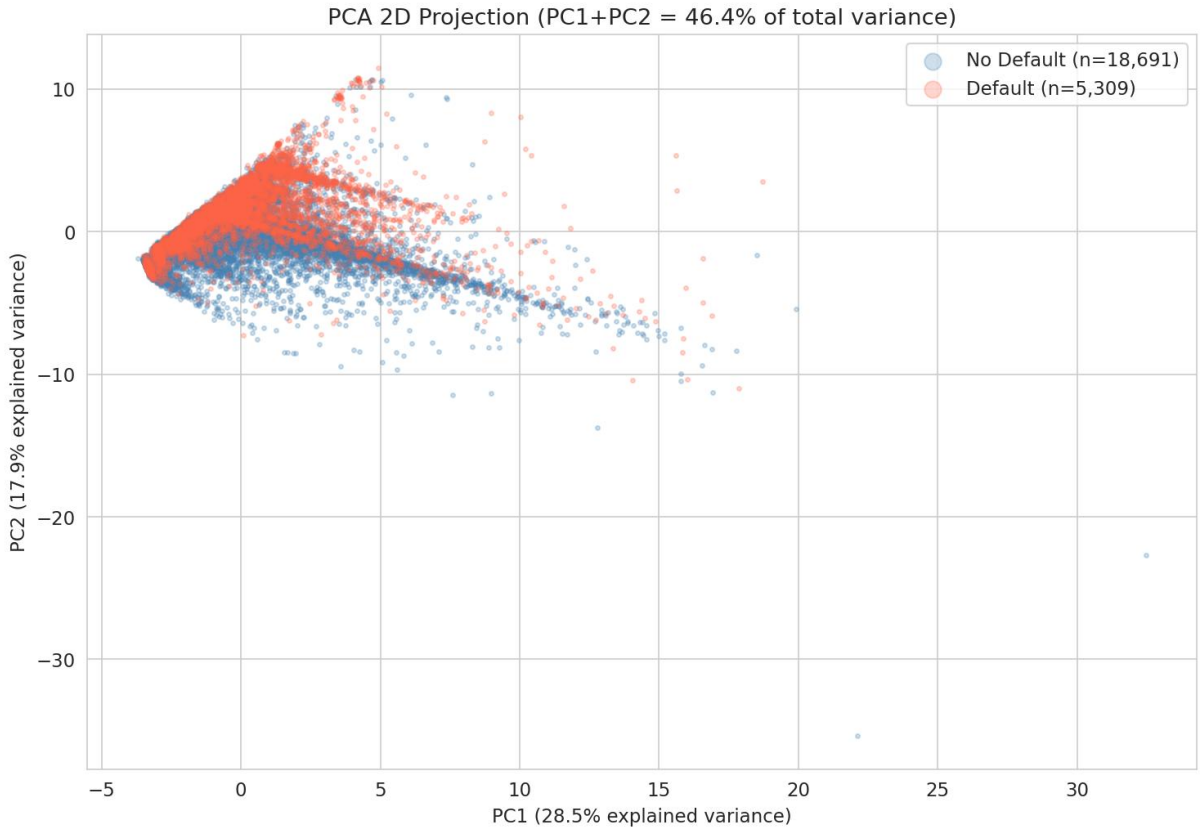
Payment History Analysis

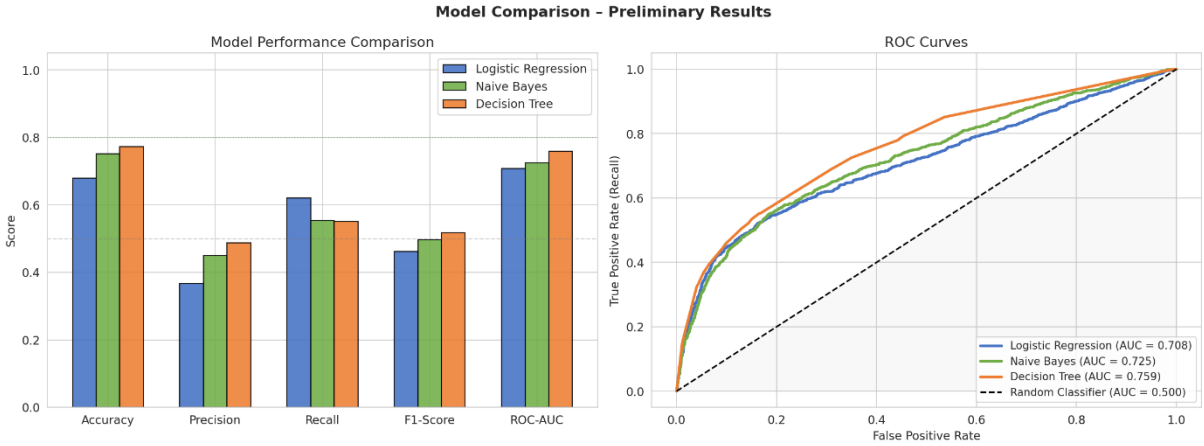


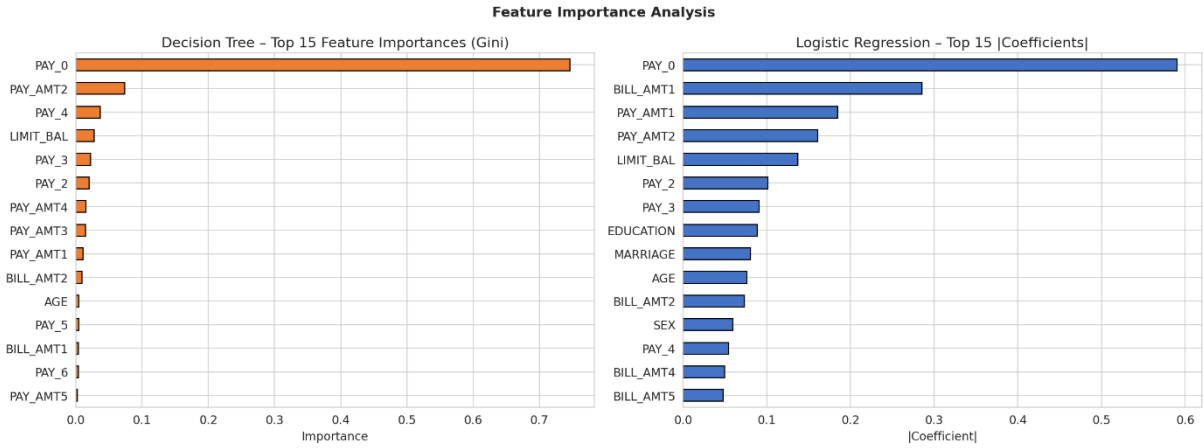
Feature Correlation Matrix (Lower Triangle)

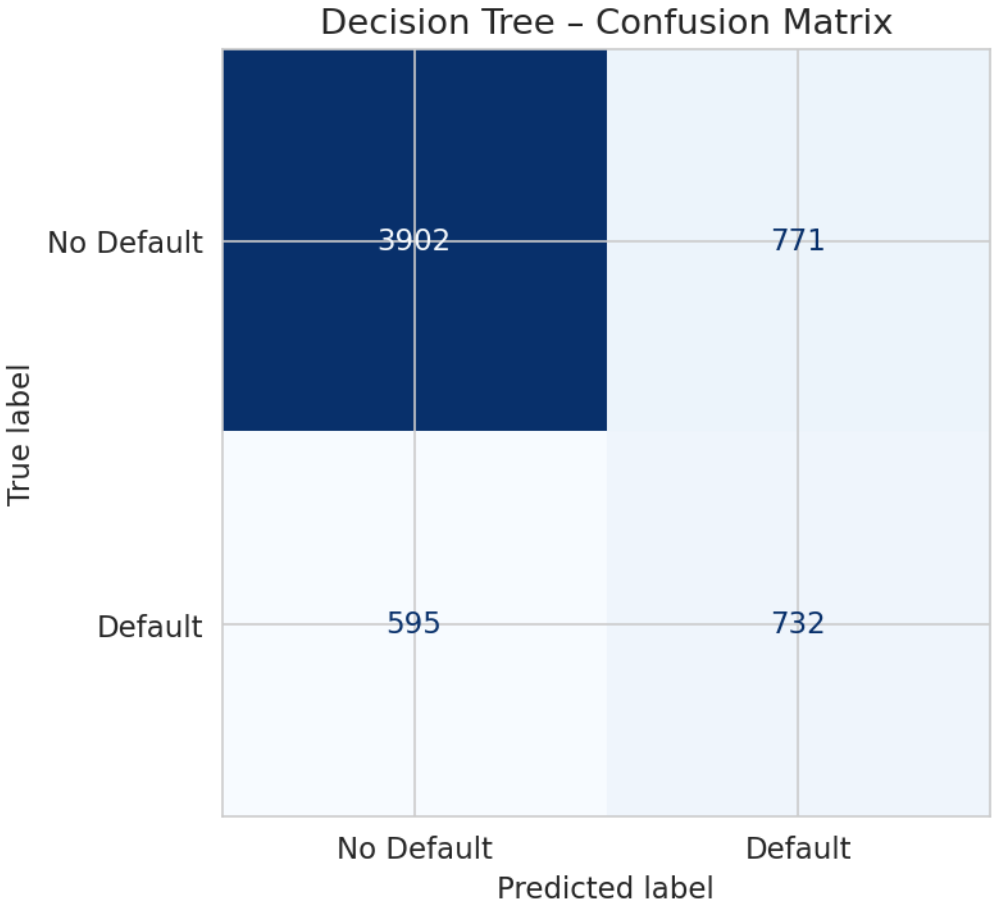


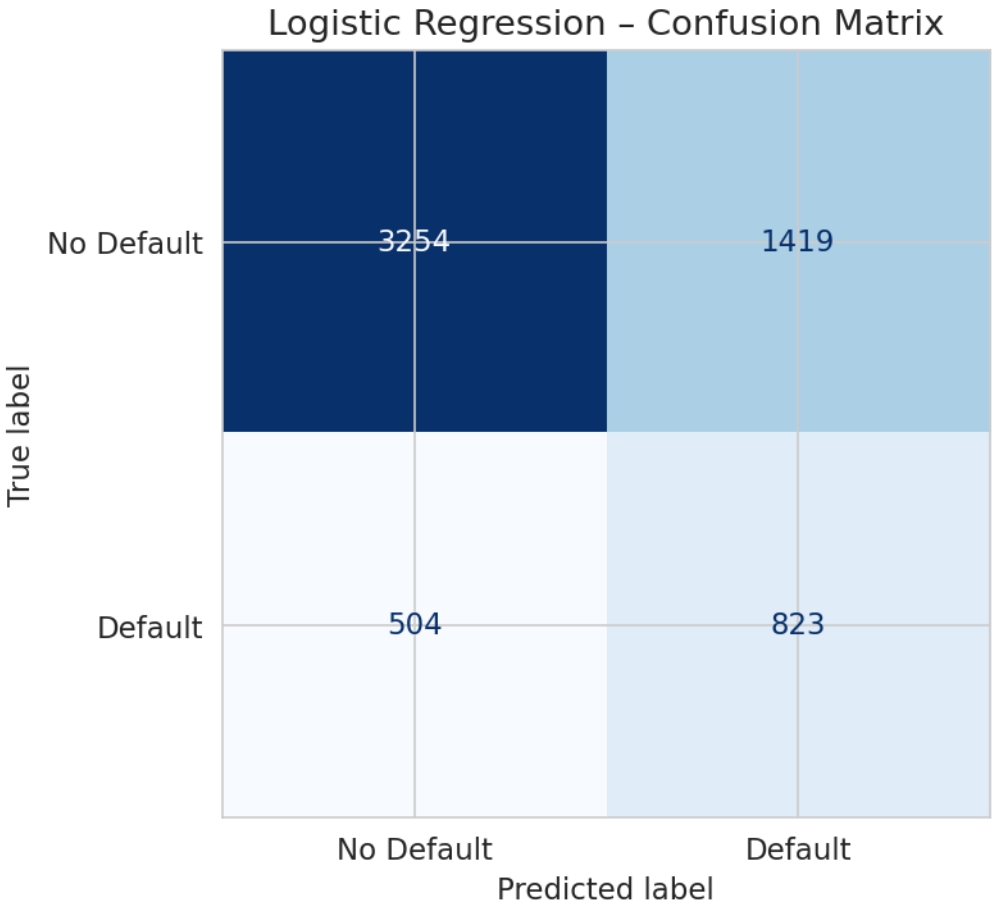

PCA - Explained Variance Analysis
Scree Plot
Explained Variance Ratio
Principal Component
Cumulative Explained Variance
Cumulative Explained Variance
Number of Components
Threshold
90%
95%
99%

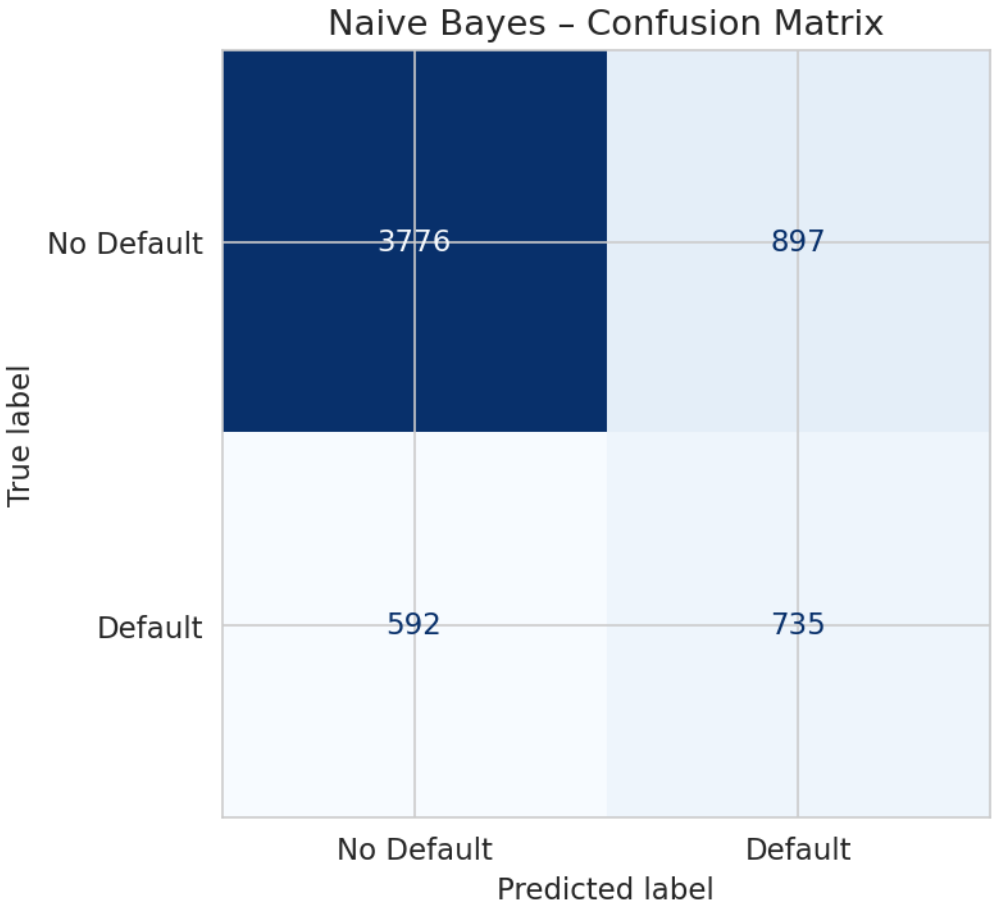

PCA 2D Projection (PC1+PC2 = 46.4% of total variance)
No Default (n=18,691)
Default (n=5,309)
PC2 (17.9% explained variance)
PC1 (28.5% explained variance)

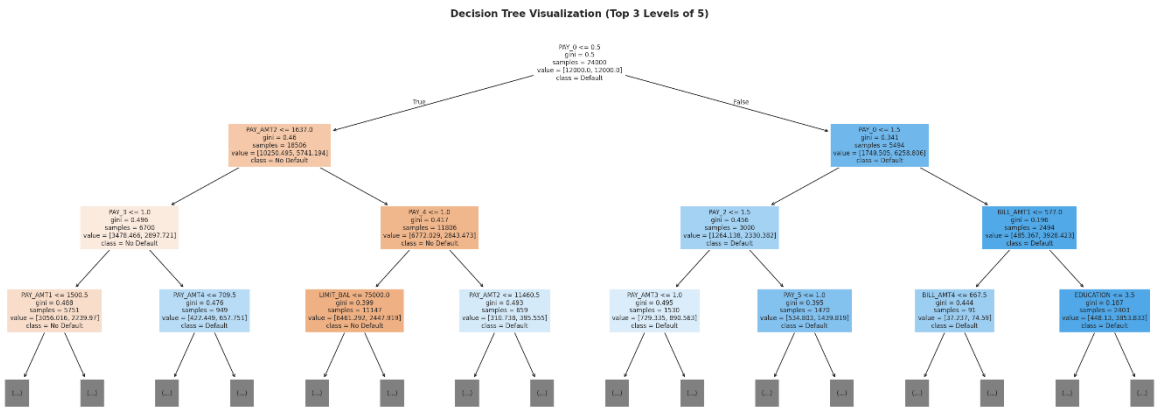

Model Comparison - Preliminary Results
Model Performance Comparison
Logistic Regression
Naive Bayes
Decision Tree
Score
Accuracy
Precision
Recall
F1-Score
ROC-AUC
ROC Curves
True Positive Rate (Recall)
False Positive Rate
Logistic Regression (AUC = 0.708)
Naive Bayes (AUC = 0.725)
Decision Tree (AUC = 0.759)
Random Classifier (AUC = 0.500)


Feature Importance Analysis
Decision Tree – Top 15 Feature Importances (Gini)
PAY_0
PAY_AMT2
PAY_4
LIMIT_BAL
PAY_3
PAY_2
PAY_AMT4
PAY_AMT3
PAY_AMT1
BILL_AMT2
AGE
PAY_5
BILL_AMT1
PAY_6
PAY_AMT5
0.0 0.1 0.2 0.3 0.4 0.5 0.6 0.7
Importance
Logistic Regression – Top 15 |Coefficients|
PAY_0
BILL_AMT1
PAY_AMT1
PAY_AMT2
LIMIT_BAL
PAY_2
PAY_3
EDUCATION
MARRIAGE
AGE
BILL_AMT2
SEX
PAY_4
BILL_AMT4
BILL_AMT5
0.0 0.1 0.2 0.3 0.4 0.5 0.6
|Coefficient|


Decision Tree – Confusion Matrix
No Default
Default
True label
3902
771
595
732
No Default
Default
Predicted label


Logistic Regression – Confusion Matrix
No Default
Default
True label
3254
1419
504
823
No Default
Default
Predicted label





12. Conclusion

We trained a Transformer neural network to serve as a spam filter. It reached an accuracy of 98% and an F1-Score / ROC-AUC of 0.9602 / 0.9966 on the test set after only 2 (effective) training epochs. As others before us observed, self-attention outperforms recurrent networks for text classification tasks. In practice, early stopping, class weights and the choice of the threshold to distinguish between positive and negative examples are crucial for good performance.

13. Raw Notebook Output

```
Libraries loaded successfully.
ucimlrepo unavailable (No module named 'ucimlrepo'). Falling back to direct
XLS download...
Dataset loaded via direct XLS download.

Dataset shape: (30000, 24)

Column names:
['LIMIT_BAL', 'SEX', 'EDUCATION', 'MARRIAGE', 'AGE', 'PAY_0', 'PAY_2',
'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6', 'BILL_AMT1', 'BILL_AMT2', 'BILL_AMT3',
'BILL_AMT4', 'BILL_AMT5', 'BILL_AMT6', 'PAY_AMT1', 'PAY_AMT2', 'PAY_AMT3',
'PAY_AMT4', 'PAY_AMT5', 'PAY_AMT6', 'default']
=== First 5 Rows ===
```

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default
II																					
1	20000	2	2	1	24	2	2	-1	-1	-2	...	0	0	0	0	689	0	0	0	0	1
2	120000	2	2	2	26	-1	2	0	0	0	...	3272	3455	3261	0	1000	1000	1000	0	2000	1
3	90000	2	2	2	34	0	0	0	0	0	...	14331	14948	15549	1518	1500	1000	1000	1000	5000	0
4	50000	2	2	1	37	0	0	0	0	0	...	28314	28959	29547	2000	2019	1200	1100	1069	1000	0
5	50000	1	2	1	57	-1	0	-1	0	0	...	20940	19146	19131	2000	36681	10000	9000	689	679	0

5 rows × 24 columns

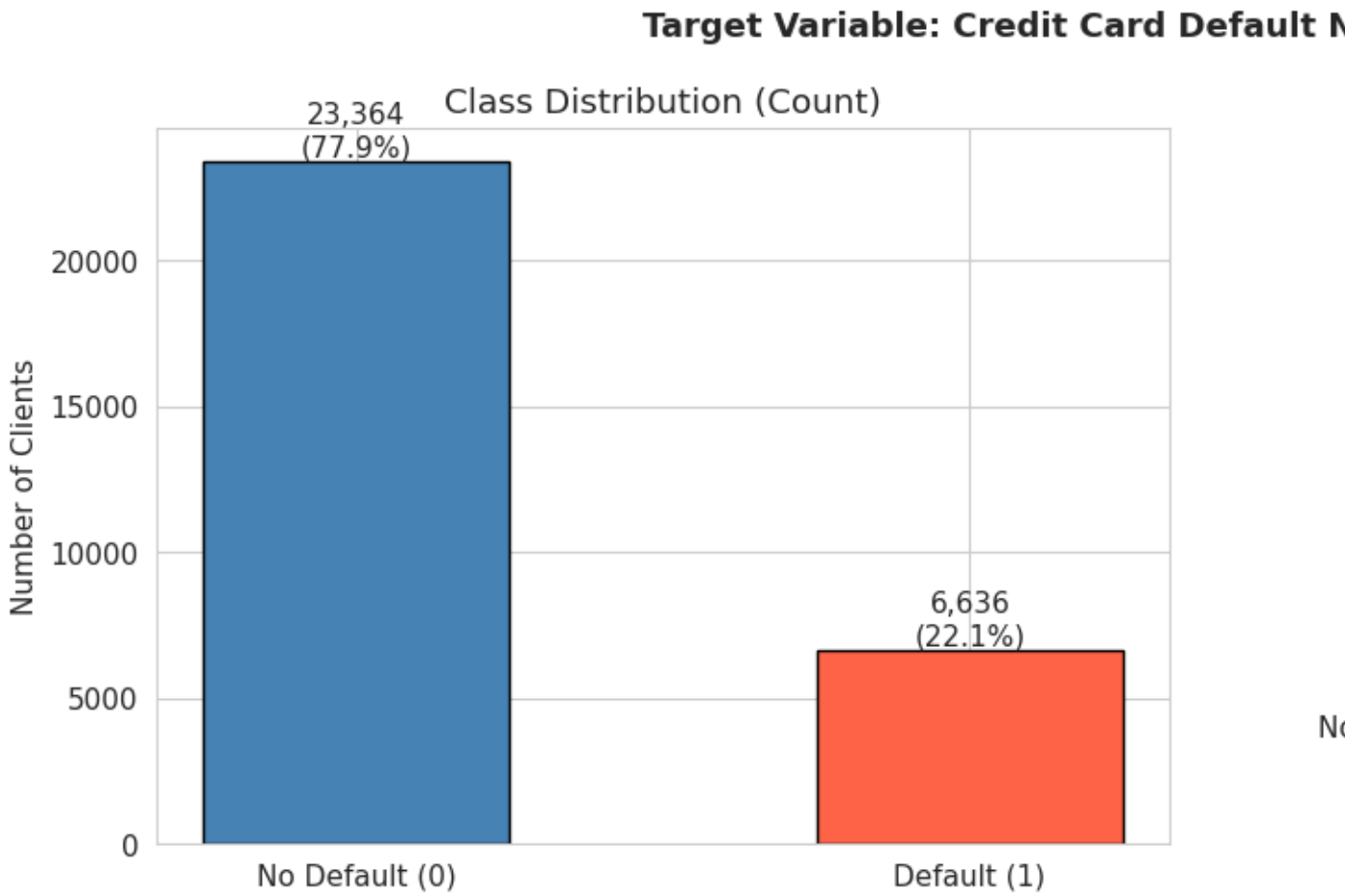
```
=== Dataset Info ===
<class 'pandas.core.frame.DataFrame'>
Index: 30000 entries, 1 to 30000
Data columns (total 24 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   LIMIT_BAL  30000 non-null  int64
 1   SEX        30000 non-null  int64
 2   EDUCATION  30000 non-null  int64
 3   MARRIAGE   30000 non-null  int64
 4   AGE        30000 non-null  int64
 5   PAY_0      30000 non-null  int64
```

```
 6   PAY_2       30000 non-null  int64
 7   PAY_3       30000 non-null  int64
 8   PAY_4       30000 non-null  int64
 9   PAY_5       30000 non-null  int64
 10  PAY_6       30000 non-null  int64
 11  BILL_AMT1   30000 non-null  int64
 12  BILL_AMT2   30000 non-null  int64
 13  BILL_AMT3   30000 non-null  int64
 14  BILL_AMT4   30000 non-null  int64
 15  BILL_AMT5   30000 non-null  int64
 16  BILL_AMT6   30000 non-null  int64
 17  PAY_AMT1    30000 non-null  int64
 18  PAY_AMT2    30000 non-null  int64
 19  PAY_AMT3    30000 non-null  int64
 20  PAY_AMT4    30000 non-null  int64
 21  PAY_AMT5    30000 non-null  int64
 22  PAY_AMT6    30000 non-null  int64
 23  default     30000 non-null  int64
dtypes: int64(24)
memory usage: 5.7 MB

=== Target Variable Distribution ===
default
0    23364
1     6636
Name: count, dtype: int64

Default rate: 22.12%
=== Statistical Summary ===
```

	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default
count	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	...	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00	30000.00
mean	167484.32	1.60	1.85	1.55	35.49	-0.02	-0.13	-0.17	-0.22	-0.27	...	43262.95	40311.40	38871.76	5663.58	5921.16	5225.68	4826.08	4799.39	5215.50	0.22
std	129747.66	0.49	0.79	0.52	9.22	1.12	1.20	1.20	1.17	1.13	...	64332.86	60797.16	59554.11	16563.28	23040.87	17606.96	15666.16	15278.31	17777.47	0.42
min	10000.00	1.00	0.00	0.00	21.00	-2.00	-2.00	-2.00	-2.00	-2.00	...	-170000.00	-81334.00	-339603.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00
25%	50000.00	1.00	1.00	1.00	28.00	-1.00	-1.00	-1.00	-1.00	-1.00	...	2326.75	1763.00	1256.00	1000.00	833.00	390.00	296.00	252.50	117.75	0.00

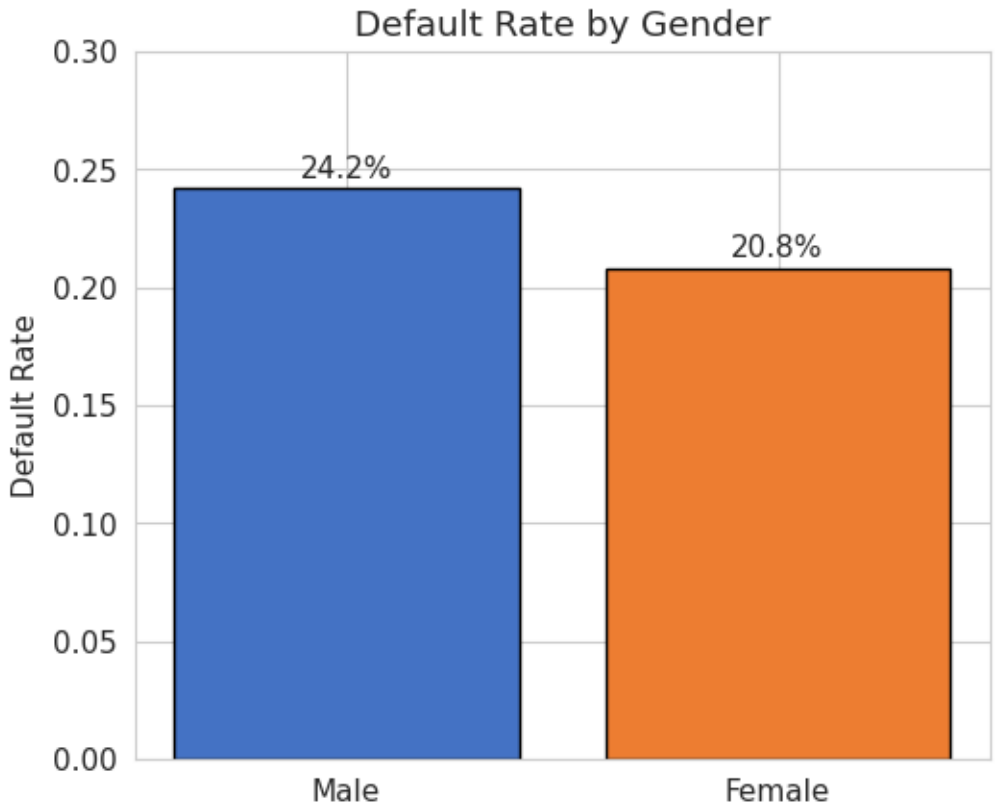
	LIMIT_BAL	SEX	EDUCATION	MARRIAGE	AGE	PAY_0	PAY_2	PAY_3	PAY_4	PAY_5	...	BILL_AMT4	BILL_AMT5	BILL_AMT6	PAY_AMT1	PAY_AMT2	PAY_AMT3	PAY_AMT4	PAY_AMT5	PAY_AMT6	default
50%	140000.00	2.00	2.00	2.00	34.00	0.00	0.00	0.00	0.00	0.00	...	19052.00	18104.50	17071.00	2100.00	2009.00	1800.00	1500.00	1500.00	1500.00	0.00
75%	240000.00	2.00	2.00	2.00	41.00	0.00	0.00	0.00	0.00	0.00	...	54506.00	50190.50	49198.25	5006.00	5000.00	4505.00	4013.25	4031.50	4000.00	0.00
max	1000000.00	2.00	6.00	3.00	79.00	8.00	8.00	8.00	8.00	8.00	...	891586.00	927171.00	961664.00	873552.00	1684259.00	896040.00	621000.00	426529.00	528666.00	1.00

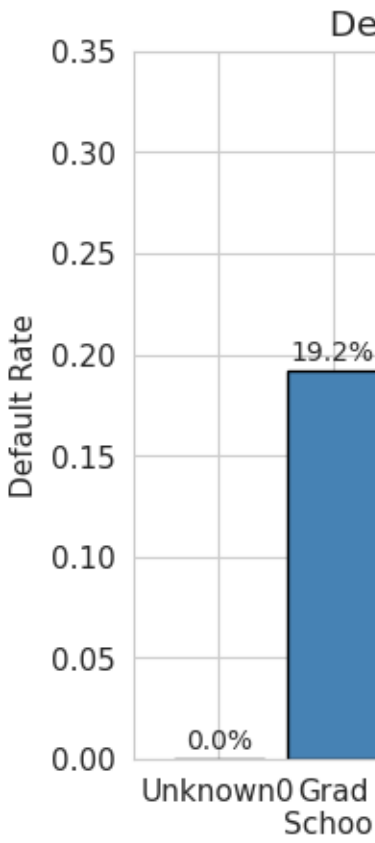
8 rows × 24 columns



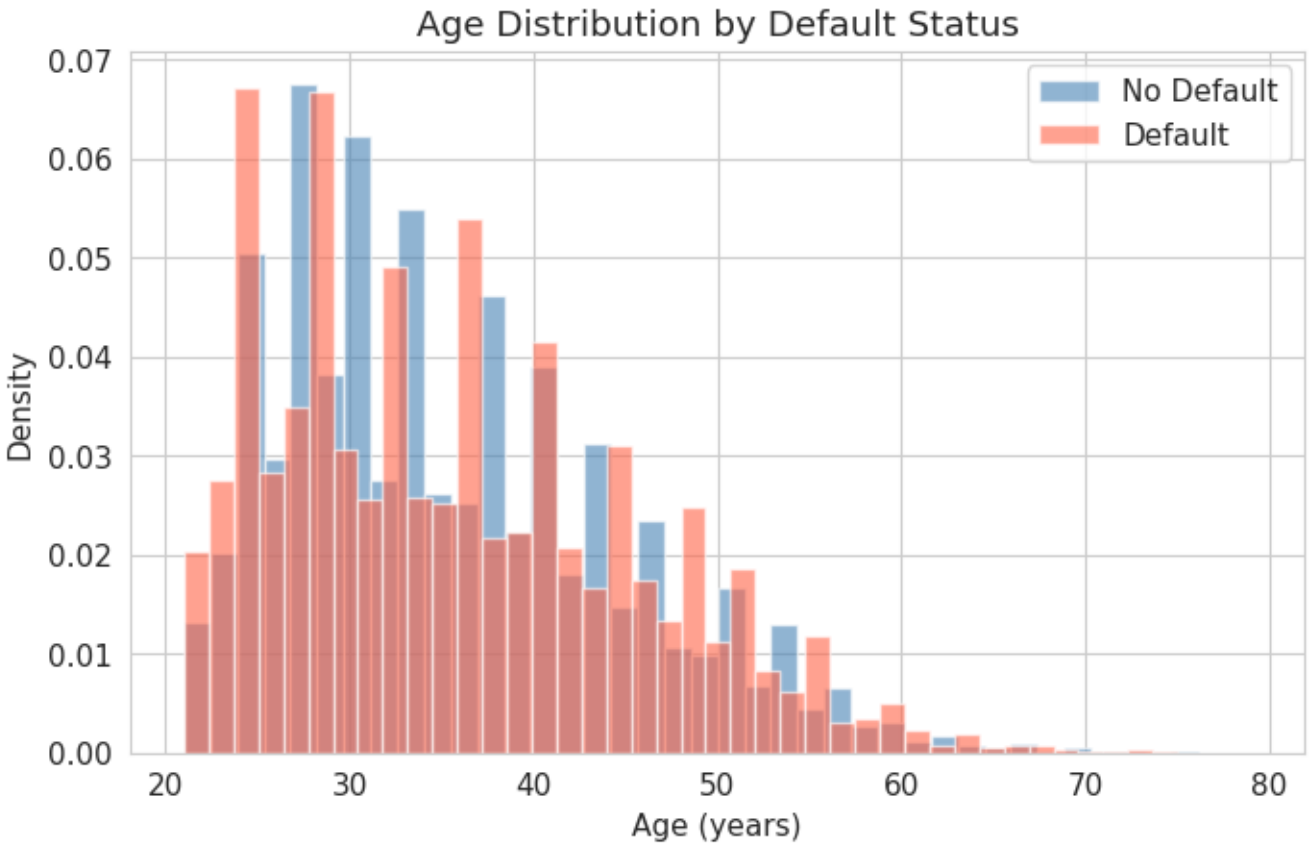

```
Class imbalance ratio: 3.5:1 (no default : default)
Note: Dataset is moderately imbalanced. class_weight="balanced" will be
used in models.
```

Default Rate by Dem





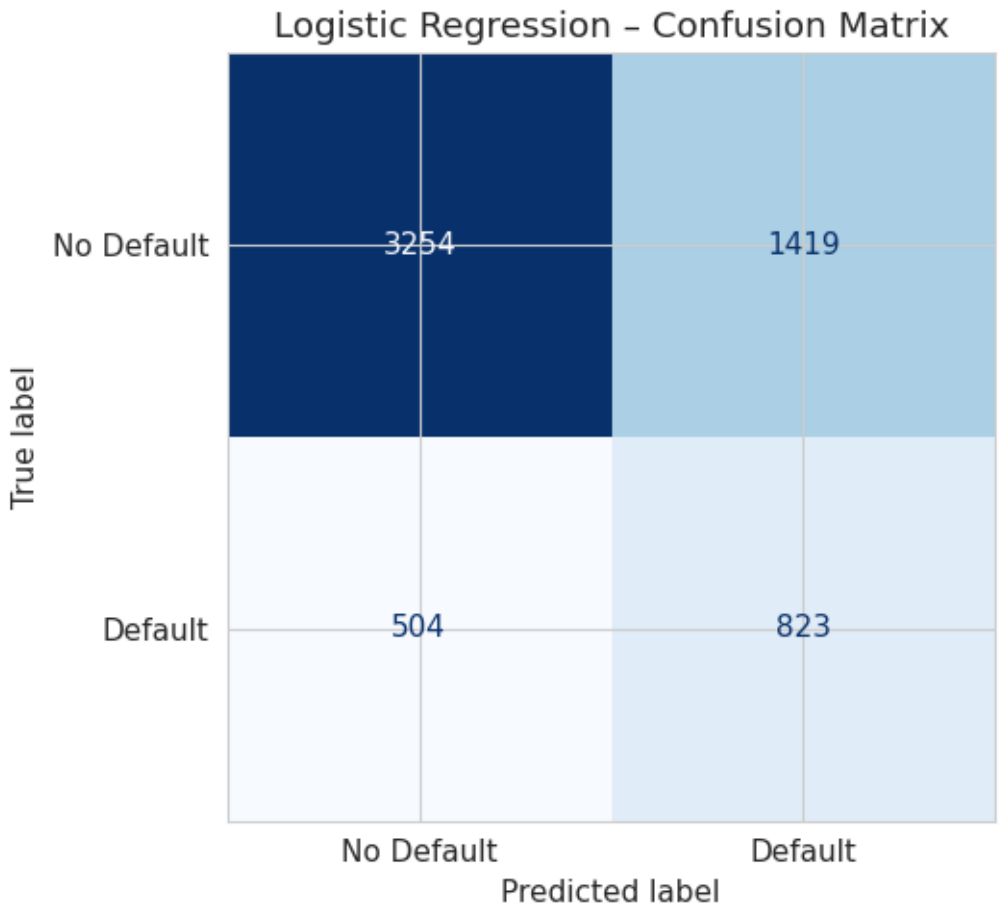
Age Analy



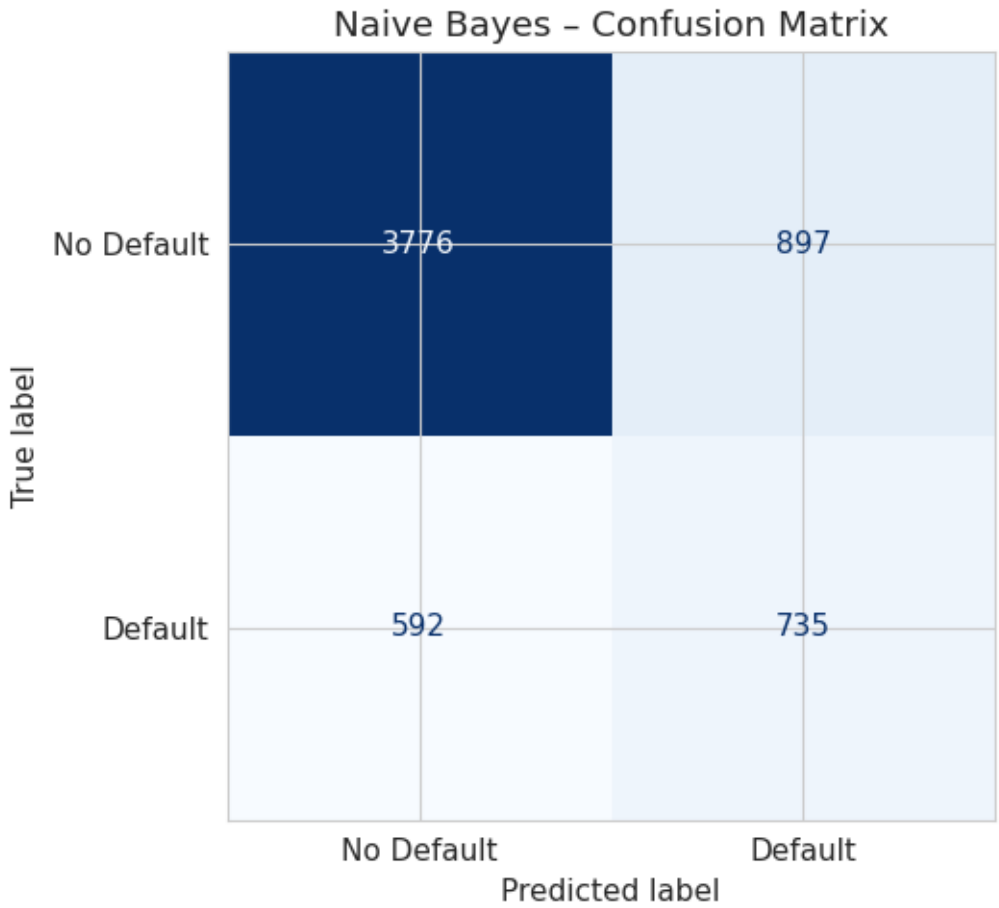

```
=== Top 10 Features Correlated with Default ===
           |Correlation|
PAY_0             0.3248
PAY_2             0.2636
PAY_3             0.2353
PAY_4             0.2166
PAY_5             0.2041
PAY_6             0.1869
LIMIT_BAL         0.1535
PAY_AMT1          0.0729
PAY_AMT2          0.0586
PAY_AMT4          0.0568
=== Missing Values ===
Total missing: 0
No missing values found. Dataset is complete.

Duplicate rows: 35

EDUCATION values (before): [np.int64(0), np.int64(1), np.int64(2),
np.int64(3), np.int64(4), np.int64(5), np.int64(6)]
Undocumented counts: 345 rows
EDUCATION values (after):  [np.int64(1), np.int64(2), np.int64(3),
np.int64(4)]

MARRIAGE values (before): [np.int64(0), np.int64(1), np.int64(2),
np.int64(3)]
Undocumented counts: 54 rows
```

```
MARRIAGE values (after):  [np.int64(1), np.int64(2), np.int64(3)]

Cleaning complete.
Feature matrix shape: (30000, 23)
Target vector shape:  (30000,)

Features (23):
   1. LIMIT_BAL
   2. SEX
   3. EDUCATION
   4. MARRIAGE
   5. AGE
   6. PAY_0
   7. PAY_2
   8. PAY_3
   9. PAY_4
  10. PAY_5
  11. PAY_6
  12. BILL_AMT1
  13. BILL_AMT2
  14. BILL_AMT3
  15. BILL_AMT4
  16. BILL_AMT5
  17. BILL_AMT6
  18. PAY_AMT1
  19. PAY_AMT2
  20. PAY_AMT3
  21. PAY_AMT4
  22. PAY_AMT5
  23. PAY_AMT6
Train size: 24,000 samples (80%)
Test size:  6,000 samples (20%)

Train default rate: 22.12%
Test  default rate: 22.12%
Stratified split confirmed: rates match.

StandardScaler fit on train set.
Train mean post-scale: 0.000000 (expected ~0)
Train std  post-scale: 1.000000  (expected ~1)
=== PCA Variance Summary ===
Total features:                23
Components for 90% variance:  13
Components for 95% variance:  15
Components for 99% variance:  19

Top 5 components explain:     64.17% of variance
Top 10 components explain:    83.09% of variance


Observation: The two classes overlap significantly in the first two PCs,
suggesting linear separability is limited - non-linear models may perform
better.
evaluate_model() helper defined.

=======================================================
  MODEL: Logistic Regression
=======================================================
              precision    recall  f1-score   support

  No Default       0.87      0.70      0.77      4673
```

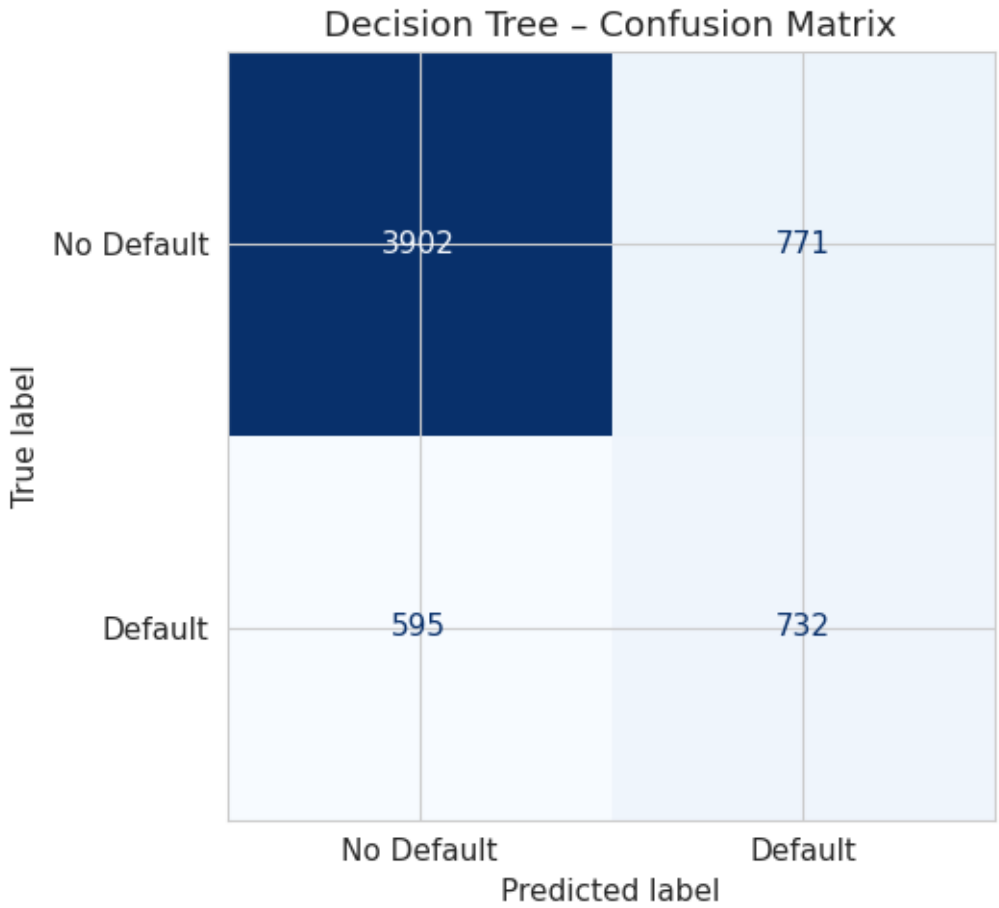
```
     Default       0.37      0.62      0.46      1327

    accuracy                           0.68      6000
   macro avg       0.62      0.66      0.62      6000
weighted avg       0.76      0.68      0.70      6000
```




```
5-Fold CV ROC-AUC: 0.7264 ± 0.0106

==========================================================
  MODEL: Naive Bayes
==========================================================
               precision    recall  f1-score   support

  No Default       0.86      0.81      0.84      4673
     Default       0.45      0.55      0.50      1327

    accuracy                           0.75      6000
   macro avg       0.66      0.68      0.67      6000
weighted avg       0.77      0.75      0.76      6000
```




```
5-Fold CV ROC-AUC: 0.7365 ± 0.0102

=========================================================
  MODEL: Decision Tree
=========================================================
              precision    recall  f1-score   support

  No Default       0.87      0.84      0.85      4673
     Default       0.49      0.55      0.52      1327

    accuracy                           0.77      6000
   macro avg       0.68      0.69      0.68      6000
weighted avg       0.78      0.77      0.78      6000
```




```
5-Fold CV ROC-AUC: 0.7577 ± 0.0067

=== Model Performance Comparison ===
```

	Accuracy	Precision	Recall	F1-Score	ROC-AUC	CV_AUC_Mean	CV_AUC_Std
Model							
Logistic Regression	0.6795	0.3671	0.6202	0.4612	0.7084	0.7264	0.0106
Naive Bayes	0.7518	0.4504	0.5539	0.4968	0.7248	0.7365	0.0102
Decision Tree	0.7723	0.4870	0.5516	0.5173	0.7589	0.7577	0.0067

```
Best F1-Score : Decision Tree (0.5173)
Best ROC-AUC  : Decision Tree (0.7589)


=== Top 10 Features: Decision Tree (Gini Importance) ===
PAY_0        0.7461
PAY_AMT2     0.0737
PAY_4        0.0372
LIMIT_BAL    0.0276
PAY_3        0.0225
PAY_2        0.0200
PAY_AMT4     0.0150
```

```
PAY_AMT3     0.0148
PAY_AMT1     0.0105
BILL_AMT2    0.0089

=== Top 10 Features: Logistic Regression (|Coefficient|) ===
PAY_0        0.5903
BILL_AMT1    0.2853
PAY_AMT1     0.1843
PAY_AMT2     0.1612
LIMIT_BAL    0.1369
PAY_2        0.1010
PAY_3        0.0909
EDUCATION    0.0887
MARRIAGE     0.0809
AGE          0.0761
==================================================================
  MILESTONE 2 - PRELIMINARY RESULTS SUMMARY
==================================================================
```

	Accuracy	Recall	F1-Score	ROC-AUC
Model				
Logistic Regression	0.6795	0.6202	0.4612	0.7084
Naive Bayes	0.7518	0.5539	0.4968	0.7248
Decision Tree	0.7723	0.5516	0.5173	0.7589

```
Key Observations:
  1. PAY_0 (most recent payment status) is the strongest predictor
  2. Payment history features (PAY_0 - PAY_6) dominate feature importance
  3. PCA shows significant class overlap in first 2 components
  4. Class imbalance (78/22) requires balanced weighting or resampling

Next Steps (for final report):
  1. Hyperparameter tuning via GridSearchCV
  2. Address class imbalance with SMOTE (imblearn)
  3. Consider ensemble models (Random Forest, Gradient Boosting)
  4. Feature engineering: payment utilization ratios (PAY_AMT/BILL_AMT)
  5. Final model selection with comprehensive analysis
```

Notebook Code:


```
{
 "cells": [
  {
   "cell_type": "markdown",
   "id": "cell_title",
   "metadata": {},
   "source": [
    "# Credit Card Default Prediction\n",
    "## BFOR 516 – Advanced Data Analytics for Cyber | Group Project Milestone 2\n",
    "\n",
    "**Dataset:** Default of Credit Card Clients (UCI ML Repository, Dataset ID 350)  \n",
    "**Source:** https://archive.ics.uci.edu/dataset/350/default+of+credit+card+clients  \n",
    "**Date:** March 2026\n",
    "\n",
    "---\n",
    "\n",
    "### Project Objective\n",
    "Predict whether a credit card client will default on their payment next month using historical demographic, credit utilization, and payment behavior data.\n",
    "\n",
    "### Models Implemented\n",
    "1. **Logistic Regression** – interpretable linear classifier; strong baseline for binary classification\n",
    "2. **Gaussian Naive Bayes** – probabilistic model; fast and effective benchmark\n",
    "3. **Decision Tree Classifier** – non-linear, rule-based model with built-in feature importance\n",
    "\n",
    "### Dataset Overview\n",
```

"- 30,000 credit card clients from Taiwan (April – September 2005)\n",

"- 23 features: demographics, credit limit, 6-month payment history, bill amounts, payment amounts\n",

"- Target: `default payment next month` (1 = default, 0 = no default)"

]

},

{

"cell_type": "code",

"execution_count": null,

"id": "cell_imports",

"metadata": {},

"outputs": [

{

"ename": "",

"evalue": "",

"output_type": "error",

"traceback": [

"\u001b[1;31mRunning cells with '.venv (Python 3.13.7)' requires the ipykernel package.\n",

"\u001b[1;31mInstall 'ipykernel' into the Python environment. \n",

"\u001b[1;31mCommand: '\"c:/Users/intel/OneDrive - University at Albany - SUNY/Documents/MSDF/Spring 2026/SP26 BFOR516 - Adv Data Analytics for Cyber/Week 2/.venv/Scripts/python.exe\" -m pip install ipykernel -U --force-reinstall'"

]

}

],

"source": [

"# ==\n",

"# 1. IMPORTS & CONFIGURATION\n",

```
"# ============================================================\n",
"import pandas as pd\n",
"import numpy as np\n",
"import matplotlib.pyplot as plt\n",
"import seaborn as sns\n",
"\n",
"from sklearn.model_selection import train_test_split, cross_val_score, StratifiedKFold\n",
"from sklearn.preprocessing import StandardScaler\n",
"from sklearn.decomposition import PCA\n",
"from sklearn.linear_model import LogisticRegression\n",
"from sklearn.naive_bayes import GaussianNB\n",
"from sklearn.tree import DecisionTreeClassifier, plot_tree\n",
"from sklearn.metrics import (\n",
"    classification_report, confusion_matrix, ConfusionMatrixDisplay,\n",
"    roc_auc_score, roc_curve,\n",
"    accuracy_score, precision_score, recall_score, f1_score\n",
")\n",
"\n",
"import warnings\n",
"warnings.filterwarnings('ignore')\n",
"\n",
"# Reproducibility\n",
"np.random.seed(42)\n",
"\n",
"# Plot defaults\n",
"plt.rcParams['figure.figsize'] = (10, 6)\n",
"plt.rcParams['font.size'] = 11\n",
"sns.set_style('whitegrid')\n",
```

"sns.set_palette('Set2')\n",
"\n",
"print('Libraries loaded successfully.')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_load",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 2. DATA LOADING\n",
"# ucimlrepo is the cleanest method; falls back to direct XLS\n",
"# ==\n",
"try:\n",
"   from ucimlrepo import fetch_ucirepo\n",
"   repo = fetch_ucirepo(id=350)\n",
"   df = pd.concat([repo.data.features, repo.data.targets], axis=1)\n",
"   # Normalize target column name\n",
"   df.columns = [c.strip() for c in df.columns]\n",
"   df = df.rename(columns={df.columns[-1]: 'default'})\n",
"   print('Dataset loaded via ucimlrepo.')\n",
"\n",
"except Exception as e:\n",
"   print(f'ucimlrepo unavailable ({e}). Falling back to direct XLS download...')\n",
"   url = (\n",

" 'https://archive.ics.uci.edu/ml/machine-learning-databases/'\n",
" '00350/default%20of%20credit%20card%20clients.xls'\n",
")\n",
" # header=1 skips the merged title row; index_col=0 drops the ID column\n",
" df = pd.read_excel(url, header=1, index_col=0)\n",
" df.columns = [c.strip() for c in df.columns]\n",
" df = df.rename(columns={'default payment next month': 'default'})\n",
" print('Dataset loaded via direct XLS download.')\n",
"\n",
"# Drop ID column if it still exists\n",
"if 'ID' in df.columns:\n",
" df = df.drop(columns=['ID'])\n",
"\n",
"print(f'\\nDataset shape: {df.shape}')\n",
"print(f'\\nColumn names:')\n",
"print(list(df.columns))"
]
},
{
"cell_type": "markdown",
"id": "cell_eda_header",
"metadata": {},
"source": [
"---\n",
"## Section 1: Exploratory Data Analysis (EDA)"
]
},
{

"cell_type": "code",
"execution_count": null,
"id": "cell_info",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 3. INITIAL DATA EXPLORATION\n",
"# ==\n",
"print('=== First 5 Rows ===')\n",
"display(df.head())\n",
"\n",
"print('\\n=== Dataset Info ===')\n",
"df.info()\n",
"\n",
"print('\\n=== Target Variable Distribution ===')\n",
"print(df['default'].value_counts())\n",
"print(f'\\nDefault rate: {df[\"default\"].mean():.2%}')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_describe",
"metadata": {},
"outputs": [],
"source": [
"# Statistical summary of all features\n",

"print('=== Statistical Summary ===')\n",
"display(df.describe().round(2))"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_class_dist",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 4. CLASS DISTRIBUTION VISUALIZATION\n",
"# ==\n",
"fig, axes = plt.subplots(1, 2, figsize=(12, 5))\n",
"\n",
"counts = df['default'].value_counts()\n",
"labels = ['No Default (0)', 'Default (1)']\n",
"colors = ['steelblue', 'tomato']\n",
"\n",
"# Bar chart\n",
"bars = axes[0].bar(labels, counts.values, color=colors, edgecolor='black', width=0.5)\n",
"axes[0].set_title('Class Distribution (Count)')\n",
"axes[0].set_ylabel('Number of Clients')\n",
"for bar, val in zip(bars, counts.values):\n",
"   axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 200,\n",
"   f'{val:,}\\n({val/len(df):.1%})', ha='center', fontsize=11)\n",
"\n",

"# Pie chart\n",
"axes[1].pie(counts.values, labels=['No Default', 'Default'],\n",
" autopct='%1.1f%%', colors=colors, startangle=90,\n",
" explode=(0, 0.06), shadow=True)\n",
"axes[1].set_title('Class Distribution (Proportion)')\n",
"\n",
"plt.suptitle('Target Variable: Credit Card Default Next Month', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_01_class_distribution.png', dpi=150, bbox_inches='tight')\n",
"plt.show()\n",
"\n",
"imbalance_ratio = counts[0] / counts[1]\n",
"print(f'Class imbalance ratio: {imbalance_ratio:.1f}:1 (no default : default)')\n",
"print('Note: Dataset is moderately imbalanced. class_weight=\"balanced\" will be used in models.')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_demographics",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 5. DEFAULT RATE BY DEMOGRAPHIC FEATURES\n",
"# ==\n",
"fig, axes = plt.subplots(1, 3, figsize=(16, 5))\n",

```
"\n",
"# SEX\n",
"sex_rate = df.groupby('SEX')['default'].mean()\n",
"sex_labels = {1: 'Male', 2: 'Female'}\n",
"axes[0].bar([sex_labels.get(k, k) for k in sex_rate.index], sex_rate.values,\n",
"           color=['#4472C4', '#ED7D31'], edgecolor='black')\n",
"axes[0].set_title('Default Rate by Gender')\n",
"axes[0].set_ylabel('Default Rate')\n",
"axes[0].set_ylim(0, 0.30)\n",
"for i, v in enumerate(sex_rate.values):\n",
"    axes[0].text(i, v + 0.005, f'{v:.1%}', ha='center', fontsize=11)\n",
"\n",
"# EDUCATION (remap after cleaning — preview with current values)\n",
"edu_rate = df.groupby('EDUCATION')['default'].mean().sort_index()\n",
"edu_labels = {1: 'Grad\\nSchool', 2: 'University', 3: 'High\\nSchool', 4: 'Other',\n",
"             5: 'Unknown5', 6: 'Unknown6', 0: 'Unknown0'}\n",
"axes[1].bar([edu_labels.get(k, str(k)) for k in edu_rate.index], edu_rate.values,\n",
"           color='steelblue', edgecolor='black')\n",
"axes[1].set_title('Default Rate by Education')\n",
"axes[1].set_ylabel('Default Rate')\n",
"axes[1].set_ylim(0, 0.35)\n",
"for i, v in enumerate(edu_rate.values):\n",
"    axes[1].text(i, v + 0.005, f'{v:.1%}', ha='center', fontsize=10)\n",
"\n",
"# MARRIAGE\n",
"mar_rate = df.groupby('MARRIAGE')['default'].mean().sort_index()\n",
"mar_labels = {0: 'Unknown', 1: 'Married', 2: 'Single', 3: 'Other'}\n",
"axes[2].bar([mar_labels.get(k, str(k)) for k in mar_rate.index], mar_rate.values,\n",
```

" color='seagreen', edgecolor='black')\n",
"axes[2].set_title('Default Rate by Marital Status')\n",
"axes[2].set_ylabel('Default Rate')\n",
"axes[2].set_ylim(0, 0.30)\n",
"for i, v in enumerate(mar_rate.values):\n",
"   axes[2].text(i, v + 0.005, f'{v:.1%}', ha='center', fontsize=11)\n",
"\n",
"plt.suptitle('Default Rate by Demographic Features (Pre-Cleaning)', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_02_demographic_default_rates.png', dpi=150, bbox_inches='tight')\n",
"plt.show()"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_age",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 6. AGE ANALYSIS\n",
"# ==\n",
"fig, axes = plt.subplots(1, 2, figsize=(14, 5))\n",
"\n",
"# Overlapping histograms by default status\n",
"axes[0].hist(df[df['default'] == 0]['AGE'], bins=40, alpha=0.6,\n",

```
"            color='steelblue', label='No Default', density=True)\n",
"axes[0].hist(df[df['default'] == 1]['AGE'], bins=40, alpha=0.6,\n",
"            color='tomato', label='Default', density=True)\n",
"axes[0].set_title('Age Distribution by Default Status')\n",
"axes[0].set_xlabel('Age (years)')\n",
"axes[0].set_ylabel('Density')\n",
"axes[0].legend()\n",
"\n",
"# Default rate by age group\n",
"bins = [20, 30, 40, 50, 60, 80]\n",
"labels_age = ['21-30', '31-40', '41-50', '51-60', '61+']\n",
"df['AGE_GROUP'] = pd.cut(df['AGE'], bins=bins, labels=labels_age)\n",
"age_default = df.groupby('AGE_GROUP', observed=False)['default'].mean()\n",
"\n",
"axes[1].bar(age_default.index.astype(str), age_default.values,\n",
"           color='steelblue', edgecolor='black')\n",
"axes[1].set_title('Default Rate by Age Group')\n",
"axes[1].set_xlabel('Age Group')\n",
"axes[1].set_ylabel('Default Rate')\n",
"axes[1].set_ylim(0, 0.35)\n",
"for i, v in enumerate(age_default.values):\n",
"    axes[1].text(i, v + 0.005, f'{v:.1%}', ha='center', fontsize=11)\n",
"\n",
"plt.suptitle('Age Analysis', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_03_age_analysis.png', dpi=150, bbox_inches='tight')\n",
"plt.show()\n",
"\n",
```

"# Remove temporary age group column\n",
"df = df.drop(columns=['AGE_GROUP'])"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_payment_history",
"metadata": {},
"outputs": [],
"source": [
"# ==========\n",
"# 7. PAYMENT HISTORY ANALYSIS (PAY_0 through PAY_6)\n",
"# PAY_0 = Sept, PAY_2 = Aug, ..., PAY_6 = April\n",
"# Values: -1=pay duly, 0=revolving credit, 1-9=months of delay\n",
"# ==========\n",
"pay_cols = ['PAY_0', 'PAY_2', 'PAY_3', 'PAY_4', 'PAY_5', 'PAY_6']\n",
"month_labels = ['Sep', 'Aug', 'Jul', 'Jun', 'May', 'Apr']\n",
"\n",
"# Average payment delay by default status\n",
"pay_defaults = df.groupby('default')[pay_cols].mean()\n",
"\n",
"fig, axes = plt.subplots(1, 2, figsize=(14, 5))\n",
"\n",
"for label, color, name in zip([0, 1], ['steelblue', 'tomato'], ['No Default', 'Default']):\n",
"   axes[0].plot(month_labels, pay_defaults.loc[label].values,\n",
"   marker='o', color=color, linewidth=2, label=name)\n",
"\n",

"axes[0].set_title('Average Payment Delay Status Over 6 Months')\n",
"axes[0].set_xlabel('Month')\n",
"axes[0].set_ylabel('Avg PAY Status (higher = more delay)')\n",
"axes[0].legend()\n",
"axes[0].axhline(0, color='gray', linestyle='--', alpha=0.5)\n",
"\n",
"# Distribution of PAY_0 (most recent payment status)\n",
"pay0_default = df.groupby('PAY_0')['default'].mean()\n",
"axes[1].bar(pay0_default.index, pay0_default.values, color='steelblue', edgecolor='black')\n",
"axes[1].set_title('Default Rate by Most Recent Payment Status (PAY_0/Sep)')\n",
"axes[1].set_xlabel('PAY_0 Value (-1=on time, 0=revolving, 1-8=months delayed)')\n",
"axes[1].set_ylabel('Default Rate')\n",
"\n",
"plt.suptitle('Payment History Analysis', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_04_payment_history.png', dpi=150, bbox_inches='tight')\n",
"plt.show()"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_corr",
"metadata": {},
"outputs": [],
"source": [
"# ==========\n",

"# 8. CORRELATION HEATMAP\n",
"# ==========\n",
"plt.figure(figsize=(16, 12))\n",
"\n",
"corr = df.corr()\n",
"mask = np.triu(np.ones_like(corr, dtype=bool)) # upper triangle mask\n",
"\n",
"sns.heatmap(corr, mask=mask, annot=False, cmap='coolwarm',\n",
"   center=0, vmin=-1, vmax=1, linewidths=0.3,\n",
"   cbar_kws={'label': 'Pearson Correlation'})\n",
"\n",
"plt.title('Feature Correlation Matrix (Lower Triangle)', fontsize=14, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_05_correlation_heatmap.png', dpi=150, bbox_inches='tight')\n",
"plt.show()\n",
"\n",
"# Top correlations with target\n",
"print('=== Top 10 Features Correlated with Default ===')\n",
"target_corr = corr['default'].drop('default').abs().sort_values(ascending=False)\n",
"print(target_corr.head(10).round(4).to_frame('|Correlation|'))"
]
},
{
"cell_type": "markdown",
"id": "cell_cleaning_header",
"metadata": {},
"source": [
"---\n",

"## Section 2: Data Cleaning & Preprocessing\n",
"\n",
"### Issues identified during EDA:\n",
"1. **EDUCATION**: Values `0`, `5`, `6` are undocumented in the original dataset paper — remapped to `4` (Others)\n",
"2. **MARRIAGE**: Value `0` is undocumented — remapped to `3` (Others)\n",
"3. **No missing values** (will confirm below)\n",
"4. **Class imbalance**: ~22% default rate — addressed via `class_weight='balanced'` in models"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_clean",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 9. DATA CLEANING\n",
"# ==\n",
"\n",
"# Check for missing values\n",
"missing = df.isnull().sum()\n",
"print('=== Missing Values ===')\n",
"print(f'Total missing: {missing.sum()}')\n",
"if missing.sum() > 0:\n",
"   print(missing[missing > 0])\n",
"else:\n",

"   print('No missing values found. Dataset is complete.')\n",
"\n",
"# Check for duplicate rows\n",
"dupes = df.duplicated().sum()\n",
"print(f'\\nDuplicate rows: {dupes}')\n",
"\n",
"# --- Fix EDUCATION: remap undocumented values (0, 5, 6) to 4 (Other) ---\n",
"print(f'\\nEDUCATION values (before): {sorted(df[\"EDUCATION\"].unique())}')\n",
"print(f'Undocumented counts: {(df[\"EDUCATION\"].isin([0, 5, 6])).sum()} rows')\n",
"df['EDUCATION'] = df['EDUCATION'].replace({0: 4, 5: 4, 6: 4})\n",
"print(f'EDUCATION values (after): {sorted(df[\"EDUCATION\"].unique())}')\n",
"\n",
"# --- Fix MARRIAGE: remap undocumented value (0) to 3 (Other) ---\n",
"print(f'\\nMARRIAGE values (before): {sorted(df[\"MARRIAGE\"].unique())}')\n",
"print(f'Undocumented counts: {(df[\"MARRIAGE\"] == 0).sum()} rows')\n",
"df['MARRIAGE'] = df['MARRIAGE'].replace({0: 3})\n",
"print(f'MARRIAGE values (after): {sorted(df[\"MARRIAGE\"].unique())}')\n",
"\n",
"print('\\nCleaning complete.')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_feature_prep",
"metadata": {},
"outputs": [],
"source": [

"# ==========\n",
"# 10. FEATURE PREPARATION\n",
"# Categorical features (SEX, EDUCATION, MARRIAGE) are kept as\n",
"# ordinal integers — appropriate for tree-based methods and\n",
"# acceptable for LR/NB given the ordinal nature of these codes.\n",
"# All features are scaled for LR and NB using StandardScaler.\n",
"# Decision Tree uses unscaled data (scale-invariant).\n",
"# ==========\n",
"\n",
"# Separate features and target\n",
"X = df.drop(columns=['default'])\n",
"y = df['default']\n",
"\n",
"print(f'Feature matrix shape: {X.shape}')\n",
"print(f'Target vector shape: {y.shape}')\n",
"print(f'\\nFeatures ({len(X.columns)}):')\n",
"for i, col in enumerate(X.columns, 1):\n",
"   print(f' {i:2d}. {col}')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_split",
"metadata": {},
"outputs": [],
"source": [
"# ==========\n",

"# 11. TRAIN / TEST SPLIT (stratified 80/20)\n",
"# Stratified split ensures the default rate is preserved\n",
"# in both train and test sets, critical for imbalanced data.\n",
"# ==\n",
"X_train, X_test, y_train, y_test = train_test_split(\n",
"   X, y, test_size=0.2, random_state=42, stratify=y\n",
")\n",
"\n",
"print(f'Train size: {X_train.shape[0]:,} samples ({X_train.shape[0]/len(X):.0%})')\n",
"print(f'Test size: {X_test.shape[0]:,} samples ({X_test.shape[0]/len(X):.0%})')\n",
"print(f'\\nTrain default rate: {y_train.mean():.2%}')\n",
"print(f'Test default rate: {y_test.mean():.2%}')\n",
"print('Stratified split confirmed: rates match.')\n",
"\n",
"# — Scaling (fit on train only to prevent data leakage) ————————————————\n",
"scaler = StandardScaler()\n",
"X_train_scaled = scaler.fit_transform(X_train)\n",
"X_test_scaled = scaler.transform(X_test) # use train statistics on test\n",
"\n",
"print(f'\\nStandardScaler fit on train set.')\n",
"print(f'Train mean post-scale: {X_train_scaled.mean():.6f} (expected ~0)')\n",
"print(f'Train std post-scale: {X_train_scaled.std():.6f} (expected ~1)')"
]
},
{
"cell_type": "markdown",
"id": "cell_pca_header",
"metadata": {},

"source": [
"---\n",
"## Section 3: Principal Component Analysis (PCA)\n",
"\n",
"PCA is used to:\n",
"1. Understand the dimensionality structure of the feature space\n",
"2. Identify how many components capture most of the variance\n",
"3. Visualize the data in 2D to check for class separability"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_pca",
"metadata": {},
"outputs": [],
"source": [
"# ==========\n",
"# 12. PCA ANALYSIS\n",
"# ==========\n",
"pca_full = PCA(random_state=42)\n",
"pca_full.fit(X_train_scaled)\n",
"\n",
"explained = pca_full.explained_variance_ratio_\n",
"cumvar = np.cumsum(explained)\n",
"\n",
"n_90 = int(np.argmax(cumvar >= 0.90)) + 1\n",
"n_95 = int(np.argmax(cumvar >= 0.95)) + 1\n",

```
"n_99 = int(np.argmax(cumvar >= 0.99)) + 1\n",
"\n",
"print('=== PCA Variance Summary ===')\n",
"print(f'Total features:              {X_train.shape[1]}')\n",
"print(f'Components for 90% variance:  {n_90}')\n",
"print(f'Components for 95% variance:  {n_95}')\n",
"print(f'Components for 99% variance:  {n_99}')\n",
"print(f'\\nTop 5 components explain:    {cumvar[4]:.2%} of variance')\n",
"print(f'Top 10 components explain:   {cumvar[9]:.2%} of variance')\n",
"\n",
"fig, axes = plt.subplots(1, 2, figsize=(14, 5))\n",
"\n",
"# Scree plot\n",
"axes[0].bar(range(1, len(explained)+1), explained, color='steelblue',\n",
"           alpha=0.8, edgecolor='white')\n",
"axes[0].plot(range(1, len(explained)+1), explained, 'o-', color='tomato',\n",
"            markersize=4, linewidth=1.5)\n",
"axes[0].set_xlabel('Principal Component')\n",
"axes[0].set_ylabel('Explained Variance Ratio')\n",
"axes[0].set_title('Scree Plot')\n",
"axes[0].set_xlim(0, 24)\n",
"\n",
"# Cumulative variance\n",
"axes[1].plot(range(1, len(cumvar)+1), cumvar, 'o-', color='steelblue',\n",
"            markersize=5, linewidth=2)\n",
"axes[1].axhline(0.90, color='green', linestyle='--', alpha=0.7, label='90%')\n",
"axes[1].axhline(0.95, color='orange', linestyle='--', alpha=0.7, label='95%')\n",
"axes[1].axhline(0.99, color='red', linestyle='--', alpha=0.7, label='99%')\n",
```

"axes[1].axvline(n_95, color='orange', linestyle=':', alpha=0.6)\n",
"axes[1].fill_between(range(1, len(cumvar)+1), cumvar, alpha=0.15, color='steelblue')\n",
"axes[1].set_xlabel('Number of Components')\n",
"axes[1].set_ylabel('Cumulative Explained Variance')\n",
"axes[1].set_title('Cumulative Explained Variance')\n",
"axes[1].legend(title='Threshold')\n",
"axes[1].set_xlim(0, 24)\n",
"\n",
"plt.suptitle('PCA – Explained Variance Analysis', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_06_pca_variance.png', dpi=150, bbox_inches='tight')\n",
"plt.show()"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_pca_scatter",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 13. PCA 2D SCATTER PLOT\n",
"# ==\n",
"pca_2d = PCA(n_components=2, random_state=42)\n",
"X_pca_train = pca_2d.fit_transform(X_train_scaled)\n",
"\n",
"plt.figure(figsize=(10, 7))\n",

"\n",
"for label, color, name in zip([0, 1], ['steelblue', 'tomato'], ['No Default', 'Default']):\n",
"   mask = y_train == label\n",
"   plt.scatter(X_pca_train[mask, 0], X_pca_train[mask, 1],\n",
"   c=color, alpha=0.25, s=6, label=f'{name} (n={mask.sum():,})')\n",
"\n",
"var1 = pca_2d.explained_variance_ratio_[0]\n",
"var2 = pca_2d.explained_variance_ratio_[1]\n",
"plt.xlabel(f'PC1 ({var1:.1%} explained variance)')\n",
"plt.ylabel(f'PC2 ({var2:.1%} explained variance)')\n",
"plt.title(f'PCA 2D Projection (PC1+PC2 = {var1+var2:.1%} of total variance)')\n",
"plt.legend(markerscale=4, fontsize=11)\n",
"plt.tight_layout()\n",
"plt.savefig('fig_07_pca_2d_scatter.png', dpi=150, bbox_inches='tight')\n",
"plt.show()\n",
"\n",
"print('Observation: The two classes overlap significantly in the first two PCs,')\n",
"print('suggesting linear separability is limited - non-linear models may perform better.')"
]
},
{
"cell_type": "markdown",
"id": "cell_models_header",
"metadata": {},
"source": [
"---\n",
"## Section 4: Machine Learning Models\n",
"\n",

"### Evaluation Metrics\n",
"- **Accuracy**: Overall correctness (misleading for imbalanced data alone)\n",
"- **Precision**: Of predicted defaults, how many are actual defaults\n",
"- **Recall**: Of actual defaults, how many were correctly identified (critical for fraud/risk)\n",
"- **F1-Score**: Harmonic mean of precision and recall\n",
"- **ROC-AUC**: Overall discriminative ability; 0.5 = random, 1.0 = perfect\n",
"- **5-Fold CV AUC**: Cross-validated AUC to assess generalization"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_helper",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 14. EVALUATION HELPER FUNCTION\n",
"# ==\n",
"all_results = []\n",
"all_probs   = {} # store test probabilities for ROC curves\n",
"\n",
"def evaluate_model(name, model, X_tr, y_tr, X_te, y_te):\n",
"   \"\"\"\n",
"   Train model, evaluate on test set, print classification report,\n",
"   display confusion matrix, and run 5-fold cross-validation.\n",
"   Returns metrics dict, fitted model, and test probabilities.\n",

```
    "    \"\"\"\n",
    "    # Train\n",
    "    model.fit(X_tr, y_tr)\n",
    "    y_pred = model.predict(X_te)\n",
    "    y_prob = model.predict_proba(X_te)[:, 1] if hasattr(model, 'predict_proba') else
None\n",
    "\n",
    "    # Metrics\n",
    "    metrics = {\n",
    "        'Model':     name,\n",
    "        'Accuracy':  round(accuracy_score(y_te, y_pred), 4),\n",
    "        'Precision': round(precision_score(y_te, y_pred, zero_division=0), 4),\n",
    "        'Recall':    round(recall_score(y_te, y_pred), 4),\n",
    "        'F1-Score':  round(f1_score(y_te, y_pred), 4),\n",
    "        'ROC-AUC':   round(roc_auc_score(y_te, y_prob), 4) if y_prob is not None else
None,\n",
    "    }\n",
    "\n",
    "    print(f'\\n{\"=\"*55}')\n",
    "    print(f'  MODEL: {name}')\n",
    "    print(f'{\"=\"*55}')\n",
    "    print(classification_report(y_te, y_pred,\n",
    "                                target_names=['No Default', 'Default']))\n",
    "\n",
    "    # Confusion matrix plot\n",
    "    fig, ax = plt.subplots(figsize=(6, 5))\n",
    "    ConfusionMatrixDisplay.from_predictions(\n",
    "        y_te, y_pred,\n",
    "        display_labels=['No Default', 'Default'],\n",
```

```
  "        cmap='Blues', ax=ax, colorbar=False\n",
  "    )\n",
  "    ax.set_title(f'{name} \\u2013 Confusion Matrix')\n",
  "    plt.tight_layout()\n",
  "    fname = name.lower().replace(' ', '_')\n",
  "    plt.savefig(f'fig_cm_{fname}.png', dpi=150, bbox_inches='tight')\n",
  "    plt.show()\n",
  "\n",
  "    # 5-Fold Stratified Cross-Validation (AUC)\n",
  "    cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)\n",
  "    cv_scores = cross_val_score(model, X_tr, y_tr, cv=cv, scoring='roc_auc')\n",
  "    metrics['CV_AUC_Mean'] = round(cv_scores.mean(), 4)\n",
  "    metrics['CV_AUC_Std']  = round(cv_scores.std(), 4)\n",
  "    print(f'5-Fold CV ROC-AUC: {cv_scores.mean():.4f} \\u00b1 {cv_scores.std():.4f}')\n",
  "\n",
  "    return metrics, model, y_prob\n",
  "\n",
  "print('evaluate_model() helper defined.')"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "cell_lr",
 "metadata": {},
 "outputs": [],
 "source": [
  "# ============================================================\n",
```

"# 15. MODEL 1: LOGISTIC REGRESSION\n",
"# ==========\n",
"# Rationale:\n",
"#   - Well-established baseline for binary classification\n",
"#   - Coefficients are directly interpretable\n",
"#   - class_weight='balanced' compensates for the 78/22 imbalance\n",
"#   - C=1.0 (default regularization); max_iter=1000 ensures convergence\n",
"# ==========\n",
"lr_model = LogisticRegression(\n",
"   C=1.0,\n",
"   max_iter=1000,\n",
"   class_weight='balanced',\n",
"   solver='lbfgs',\n",
"   random_state=42\n",
")\n",
"\n",
"lr_metrics, trained_lr, prob_lr = evaluate_model(\n",
"   'Logistic Regression', lr_model,\n",
"   X_train_scaled, y_train,\n",
"   X_test_scaled, y_test\n",
")\n",
"\n",
"all_results.append(lr_metrics)\n",
"all_probs['Logistic Regression'] = prob_lr"
]
},
{
"cell_type": "code",

"execution_count": null,
"id": "cell_nb",
"metadata": {},
"outputs": [],
"source": [
"# ==========\n",
"# 16. MODEL 2: GAUSSIAN NAIVE BAYES\n",
"# ==========\n",
"# Rationale:\n",
"#   - Probabilistic model based on Bayes theorem\n",
"#   - Assumes feature independence (naive assumption)\n",
"#   - Fast training; useful as a probabilistic benchmark\n",
"#   - GaussianNB assumes continuous features follow a normal\n",
"#   distribution per class — reasonable after StandardScaler\n",
"# ==========\n",
"nb_model = GaussianNB()\n",
"\n",
"nb_metrics, trained_nb, prob_nb = evaluate_model(\n",
"   'Naive Bayes', nb_model,\n",
"   X_train_scaled, y_train,\n",
"   X_test_scaled, y_test\n",
")\n",
"\n",
"all_results.append(nb_metrics)\n",
"all_probs['Naive Bayes'] = prob_nb"
]
},
{

"cell_type": "code",
"execution_count": null,
"id": "cell_dt",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 17. MODEL 3: DECISION TREE CLASSIFIER\n",
"# ==\n",
"# Rationale:\n",
"#   - Non-linear model; can capture complex interaction patterns\n",
"#   - Built-in feature importance via Gini impurity\n",
"#   - Highly interpretable — can visualize decision rules\n",
"#   - Does NOT require feature scaling\n",
"#   - max_depth=5 prevents overfitting on 30k rows\n",
"#   - min_samples_leaf=20 ensures stable leaf nodes\n",
"# ==\n",
"dt_model = DecisionTreeClassifier(\n",
"   max_depth=5,\n",
"   min_samples_split=50,\n",
"   min_samples_leaf=20,\n",
"   class_weight='balanced',\n",
"   criterion='gini',\n",
"   random_state=42\n",
")\n",
"\n",
"# Use unscaled data for Decision Tree\n",
"dt_metrics, trained_dt, prob_dt = evaluate_model(\n",

```
  "    'Decision Tree', dt_model,\n",
  "    X_train.values, y_train,\n",
  "    X_test.values,  y_test\n",
  ")\n",
  "\n",
  "all_results.append(dt_metrics)\n",
  "all_probs['Decision Tree'] = prob_dt"
 ]
},
{
 "cell_type": "code",
 "execution_count": null,
 "id": "cell_dt_viz",
 "metadata": {},
 "outputs": [],
 "source": [
  "# Visualize Decision Tree (first 3 levels for readability)\n",
  "plt.figure(figsize=(22, 8))\n",
  "plot_tree(\n",
  "    trained_dt,\n",
  "    feature_names=X.columns.tolist(),\n",
  "    class_names=['No Default', 'Default'],\n",
  "    filled=True, max_depth=3,\n",
  "    fontsize=8, impurity=True, proportion=False\n",
  ")\n",
  "plt.title('Decision Tree Visualization (Top 3 Levels of 5)',\n",
  "          fontsize=13, fontweight='bold')\n",
  "plt.tight_layout()\n",
```

"plt.savefig('fig_decision_tree_viz.png', dpi=150, bbox_inches='tight')\n",
"plt.show()"
]
},
{
"cell_type": "markdown",
"id": "cell_results_header",
"metadata": {},
"source": [
"---\n",
"## Section 5: Model Comparison & Analysis"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_comparison",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 18. MODEL COMPARISON TABLE\n",
"# ==\n",
"results_df = pd.DataFrame(all_results).set_index('Model')\n",
"display_cols = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC',\n",
" 'CV_AUC_Mean', 'CV_AUC_Std']\n",
"results_df = results_df[display_cols]\n",
"\n",

"print('=== Model Performance Comparison ===')\n",
"display(results_df)\n",
"\n",
"best_f1 = results_df['F1-Score'].idxmax()\n",
"best_auc = results_df['ROC-AUC'].idxmax()\n",
"print(f'\\nBest F1-Score : {best_f1} ({results_df.loc[best_f1, \"F1-Score\"]})')\n",
"print(f'Best ROC-AUC : {best_auc} ({results_df.loc[best_auc, \"ROC-AUC\"]})')"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_bar_roc",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 19. COMPARISON BAR CHART + ROC CURVES\n",
"# ==\n",
"fig, axes = plt.subplots(1, 2, figsize=(16, 6))\n",
"model_colors = ['#4472C4', '#70AD47', '#ED7D31']\n",
"\n",
"# --- Bar chart of key metrics ---\n",
"metrics_to_plot = ['Accuracy', 'Precision', 'Recall', 'F1-Score', 'ROC-AUC']\n",
"x = np.arange(len(metrics_to_plot))\n",
"width = 0.22\n",
"\n",
"for i, (model_name, row) in enumerate(results_df.iterrows()):\n",

```
"    vals = [row[m] for m in metrics_to_plot]\n",
"    axes[0].bar(x + i*width, vals, width, label=model_name,\n",
"                color=model_colors[i], edgecolor='black', alpha=0.88)\n",
"\n",
"axes[0].set_xticks(x + width)\n",
"axes[0].set_xticklabels(metrics_to_plot)\n",
"axes[0].set_ylim(0, 1.05)\n",
"axes[0].set_ylabel('Score')\n",
"axes[0].set_title('Model Performance Comparison')\n",
"axes[0].legend()\n",
"axes[0].axhline(0.5, color='gray', linestyle='--', alpha=0.4, linewidth=1)\n",
"axes[0].axhline(0.8, color='green', linestyle=':', alpha=0.4, linewidth=1)\n",
"\n",
"# --- ROC Curves ---\n",
"for (model_name, prob), color in zip(all_probs.items(), model_colors):\n",
"    if prob is not None:\n",
"        fpr, tpr, _ = roc_curve(y_test, prob)\n",
"        auc = roc_auc_score(y_test, prob)\n",
"        axes[1].plot(fpr, tpr, color=color, lw=2.5,\n",
"                     label=f'{model_name} (AUC = {auc:.3f})')\n",
"\n",
"axes[1].plot([0, 1], [0, 1], 'k--', lw=1.5, label='Random Classifier (AUC = 0.500)')\n",
"axes[1].fill_between([0, 1], [0, 1], alpha=0.05, color='gray')\n",
"axes[1].set_xlabel('False Positive Rate')\n",
"axes[1].set_ylabel('True Positive Rate (Recall)')\n",
"axes[1].set_title('ROC Curves')\n",
"axes[1].legend(loc='lower right', fontsize=10)\n",
"\n",
```

"plt.suptitle('Model Comparison – Preliminary Results', fontsize=14, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_08_model_comparison_roc.png', dpi=150, bbox_inches='tight')\n",
"plt.show()"
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_feat_imp",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 20. FEATURE IMPORTANCE ANALYSIS\n",
"# ==\n",
"fig, axes = plt.subplots(1, 2, figsize=(16, 6))\n",
"\n",
"# --- Decision Tree: Gini Feature Importance ---\n",
"dt_importance = pd.Series(\n",
"   trained_dt.feature_importances_,\n",
"   index=X.columns\n",
").sort_values(ascending=True).tail(15)\n",
"\n",
"dt_importance.plot(kind='barh', color='#ED7D31', edgecolor='black', ax=axes[0])\n",
"axes[0].set_title('Decision Tree \\u2013 Top 15 Feature Importances (Gini)')\n",
"axes[0].set_xlabel('Importance')\n",
"\n",

"# --- Logistic Regression: |Coefficient| ---\n",
"lr_importance = pd.Series(\n",
"   np.abs(trained_lr.coef_[0]),\n",
"   index=X.columns\n",
").sort_values(ascending=True).tail(15)\n",
"\n",
"lr_importance.plot(kind='barh', color='#4472C4', edgecolor='black', ax=axes[1])\n",
"axes[1].set_title('Logistic Regression \\u2013 Top 15 |Coefficients|')\n",
"axes[1].set_xlabel('|Coefficient|')\n",
"\n",
"plt.suptitle('Feature Importance Analysis', fontsize=13, fontweight='bold')\n",
"plt.tight_layout()\n",
"plt.savefig('fig_09_feature_importance.png', dpi=150, bbox_inches='tight')\n",
"plt.show()\n",
"\n",
"print('=== Top 10 Features: Decision Tree (Gini Importance) ===')\n",
"print(dt_importance.sort_values(ascending=False).head(10).round(4).to_string())\n",
"\n",
"print('\\n=== Top 10 Features: Logistic Regression (|Coefficient|) ===')\n",
"print(lr_importance.sort_values(ascending=False).head(10).round(4).to_string())"
]
},
{
"cell_type": "markdown",
"id": "cell_summary_header",
"metadata": {},
"source": [
"---\n",

"## Section 6: Summary of Preliminary Results\n",
"\n",
"> **Note for report writing:** Fill in the actual numbers after running this notebook. The analysis and interpretation below must be written in your own words for the written report."
]
},
{
"cell_type": "code",
"execution_count": null,
"id": "cell_summary",
"metadata": {},
"outputs": [],
"source": [
"# ==\n",
"# 21. SUMMARY PRINTOUT\n",
"# ==\n",
"print('=' * 65)\n",
"print(' MILESTONE 2 – PRELIMINARY RESULTS SUMMARY')\n",
"print('=' * 65)\n",
"\n",
"summary_cols = ['Accuracy', 'Recall', 'F1-Score', 'ROC-AUC']\n",
"display(results_df[summary_cols])\n",
"\n",
"print('\\nKey Observations:')\n",
"print(' 1. PAY_0 (most recent payment status) is the strongest predictor')\n",
"print(' 2. Payment history features (PAY_0 - PAY_6) dominate feature importance')\n",
"print(' 3. PCA shows significant class overlap in first 2 components')\n",
"print(' 4. Class imbalance (78/22) requires balanced weighting or resampling')\n",

"\n",
"print('\\nNext Steps (for final report):')\n",
"print(' 1. Hyperparameter tuning via GridSearchCV')\n",
"print(' 2. Address class imbalance with SMOTE (imblearn)')\n",
"print(' 3. Consider ensemble models (Random Forest, Gradient Boosting)')\n",
"print(' 4. Feature engineering: payment utilization ratios (PAY_AMT/BILL_AMT)')\n",
"print(' 5. Final model selection with comprehensive analysis')"
]
}
],
"metadata": {
"kernelspec": {
"display_name": ".venv",
"language": "python",
"name": "python3"
},
"language_info": {
"codemirror_mode": {
"name": "ipython",
"version": 3
},
"file_extension": ".py",
"mimetype": "text/x-python",
"name": "python",
"nbformat_minor": 5,
"version": "3.13.7"
}
},

"nbformat": 4,

"nbformat_minor": 5

}